Syrian Private University

Faculty of Artificial Intelligence Engineering

Semester Project

Detecting Data Leakage in Cloud Storage with Interpretable Machine Learning

Prepared by:

Ghaith Jbili

Bayan Awad

Mohammed Ghazal

Supervised by:

Dr. Wassim Janeidi

Eng. Jaafar Mahfouz

Academic Year 2026–2027

Abstract

This section will be completed at the end of the semester, once the implementation is finished and the experimental results of the project are available.

Table of Contents

Abstract ..... **i**

List of Figures ..... **iv**

List of Tables ..... **v**

List of Abbreviations ..... **vi**

List of Symbols ..... **viii**

Chapter 1: Project Overview and Theoretical Foundations **1**

Introduction 1

Importance of the general field: Artificial Intelligence 1

Importance of the specific field: Data Loss Prevention in cloud computing 1

Scientific Problem 2

Project Objectives 3

Summary of the reference article 3

What this project aims to implement 4

Related Works 4

Study 1: Data leakage risks in machine learning (Apicella et al., 2025) 5

Study 2: ARKAIV, predicting data exfiltration (Hakim et al., 2025) 5

Study 3: Insider-threat detection in cloud environments (Komka et al., 2026) 6

Summary of the related works 6

Practical Applications of the Project 7

Scientific Challenges 8

Cloud Computing and Cloud Storage 8

Definition and essential characteristics 8

Service models and the shared-responsibility principle 9

Cloud storage and its threat landscape 9

Information Security and Data Leakage 10

Confidentiality and data leakage 10

Sources of data leakage and the insider threat 10

Anatomy of a leakage event in cloud storage 11

Threat model of the project 12

Data Loss Prevention 13

Concept and the three states of data 13

Functional architecture of a DLP system 13

Rule-based and learning-based detection 14

Identity, Authentication and Access Control 14

Machine Learning for Leakage Detection 15

Learning paradigms 15

Formulation of the detection task 15

Data Preprocessing and the Limits of the Dataset 16

Missing values 16

Outliers and the interquartile-range rule 16

Encoding and scaling 16

Duplicate events and the performance ceiling 17

Decision Trees 18

Structure and rules 18

Choosing splits: entropy, Gini and information gain....18
Overfitting, pruning and hyper-parameters....19
Feature importance....20
Tree Ensembles....20
Learning from Imbalanced Data....20
Resampling and SMOTE....21
Class weights and threshold moving....21
Model Selection and Hyper-parameter Tuning....21
Evaluation Metrics....21
Confusion matrix and basic metrics....21
Metrics robust to imbalance....22
Applying the metrics to the reference article....23
Methodological Data Leakage in Machine Learning....23
From Model to Cloud Service....25
Cloud-storage audit logs....25
Prototype architecture....26
Monitoring and retraining....27
Chapter Summary....28
Chapter 2: Implementation Mechanism and Experimental Results....29
Implementation Mechanism....29
Notation....29
Block diagram....29
Flowcharts....30
Pseudo-code....32
Experimental Setup....34
Experimental Results....35
Data audit....35
Performance ceiling....36
The reference tree under a leakage-safe evaluation....36
Tuning the tree on the nine attributes....37
Adding External Destination and the hour....37
Model comparison....37
Imbalance handling....38
Choosing the alert threshold....38
Final evaluation on the test set....39
Interpretable rules and attribute importance....39
Discussion and Limitations....41
Chapter Summary....41
References....42

List of Figures

Figure 1: Global average cost of a data breach, 2021–2026....2
Figure 2: Proposed leakage-aware detection framework....4
Figure 3: Division of responsibility across cloud service models....9
Figure 4: Stages of a leakage event in cloud storage and the related dataset attributes....11
Figure 5: The three states of data and typical DLP controls for each....13
Figure 6: Functional architecture of a data loss prevention system....14
Figure 7: Example of a decision tree for leakage detection....18
Figure 8: Confusion matrix of the optimised decision tree in the reference article....22
Figure 9: Taxonomy of methodological data leakage....24
Figure 10: Leaky and leakage-safe orderings of pipeline steps....24
Figure 11: Planned prototype: from cloud-storage audit events to alerts....27
Figure 12: Block diagram of the implementation mechanism....30
Figure 13: Flowchart of model training and evaluation (offline phase)....31
Figure 14: Flowchart of online scoring of audit events (planned prototype)....32
Figure 15: Leakage rate by hour of the event....36
Figure 16: Confusion matrix of the final tree and precision-recall curves on the test set....39

List of Tables

Table 1: Key results of the reference article and our recomputation....3
Table 2: Summary of the selected related works....6
Table 3: Scientific challenges and the planned response....8
Table 4: Main sources of data leakage in cloud storage....11
Table 5: Threat model: adversaries and coverage by the detector....12
Table 6: Mapping of the targeted behaviours to MITRE ATT&CK techniques....12
Table 7: Rule-based versus machine-learning-based leakage detection....14
Table 8: Authentication factors and the corresponding dataset attributes....15
Table 9: Illustrative calculation of the performance ceiling for four attribute combinations....17
Table 10: Worked example of split evaluation with entropy and the Gini index....19
Table 11: Main hyper-parameters of scikit-learn's DecisionTreeClassifier....20
Table 12: Metrics recomputed from the confusion matrix of the reference article....23
Table 13: Leakage types and the safe practice adopted in this project....25
Table 14: Possible mapping from cloud audit-log fields to the dataset attributes....26
Table 15: Planned request and response of the scoring API....27
Table 16: Attributes used in the experiments....34
Table 17: Steps of the notebook and their outputs....35
Table 18: Performance ceiling of the attribute sets....36
Table 19: Reference tree and tuned tree on the nine attributes (5-fold cross-validation)....37
Table 20: Effect of the attribute set on the tuned tree (5-fold cross-validation)....37
Table 21: Comparison of the three models on the eleven attributes (5-fold cross-validation)....38
Table 22: Imbalance strategies on the selected tree (5-fold cross-validation)....38
Table 23: Operating points of the final tree (out-of-fold scores, training set)....38
Table 24: Results on the locked test set (9,900 events, 3,079 of them leakage)....39
Table 25: The five rules that catch the most leakage events....40
Table 26: Permutation importance of the attributes on the test set....40

List of Abbreviations

Abbreviation	Full term
ABAC	Attribute-Based Access Control
AI	Artificial Intelligence
AP	Average Precision
API	Application Programming Interface
ATT&CK	Adversarial Tactics, Techniques and Common Knowledge (MITRE framework)
AWS	Amazon Web Services
CART	Classification and Regression Trees
CASB	Cloud Access Security Broker
CERT	Computer Emergency Response Team (CERT Division, Carnegie Mellon University)
CIA	Confidentiality, Integrity and Availability
DLP	Data Loss Prevention (also Data Leakage Prevention)
DT	Decision Tree
ENN	Edited Nearest Neighbours
FN	False Negative
FP	False Positive
GDPR	General Data Protection Regulation
HTTP	Hypertext Transfer Protocol
IaaS	Infrastructure as a Service
IAM	Identity and Access Management
IBM	International Business Machines Corporation
ID3	Iterative Dichotomiser 3
IEEE	Institute of Electrical and Electronics Engineers
IQR	Interquartile Range
JSON	JavaScript Object Notation
MCC	Matthews Correlation Coefficient
MDI	Mean Decrease in Impurity
MFA	Multi-Factor Authentication
ML	Machine Learning
NIST	National Institute of Standards and Technology
NJESTI	Nature Journal of Emerging Sciences, Technologies, & Innovations
PaaS	Platform as a Service
PIN	Personal Identification Number
PR	Precision-Recall
RBAC	Role-Based Access Control
RF	Random Forest
ROC	Receiver Operating Characteristic
SaaS	Software as a Service
SHAP	SHapley Additive exPlanations
SIEM	Security Information and Event Management

Abbreviation	Full term
SMOTE	Synthetic Minority Over-sampling Technique
SMOTE-NC	SMOTE for Nominal and Continuous features
SMOTEN	SMOTE for Nominal features
SOC	Security Operations Centre
SVM	Support Vector Machine
TN	True Negative
TP	True Positive
UEBA	User and Entity Behaviour Analytics
XGBoost	eXtreme Gradient Boosting

List of Symbols

Symbol	Meaning	First used in
$\mathbf{x}$	Feature vector describing one activity event	Equation (1)
y	True class label (0 = normal, 1 = leakage)	Text
$\hat{y}$	Predicted class label	Equation (1)
$s(\mathbf{x})$	Leakage score (estimated probability of leakage) produced by a classifier	Equation (1)
θ	Decision threshold applied to the score	Equation (1)
D	Dataset of labelled events	Text
D_{tr}, D_{te}	Training and test subsets of the dataset	Text
θ^*	Decision threshold selected for the final model	Text
r^*	Target recall for the leakage class	Text
ε	Tolerance within which the decision tree is preferred to a more complex model	Text
P^*	Final trained pipeline	Text
Q_1, Q_3	First and third quartiles of a feature	Equation (2)
x, x'	Original value and min-max scaled value of a feature	Equation (3)
$x_{\min}$, $x_{\max}$	Minimum and maximum of a feature in the training data	Equation (3)
n	Number of events (samples)	Equation (4)
V	Set of distinct attribute combinations observed in the data	Equation (4)
v	One attribute combination (Eq. 4) or one value of an attribute (Eq. 7)	Equation (4)
n_v, m_v	Number of events with combination v and number of those labelled as leakage	Equation (4)
Acc^*	Bayes-optimal (highest achievable) accuracy	Equation (4)
c	Share of events whose attribute combination appears with both labels	Text
S	Set of training samples reaching a tree node	Equation (5)
K	Number of classes	Equation (5)
k	Class index; also the number of folds in k-fold cross-validation	Equation (5)
p_k	Proportion of samples of class k in a node	Equation (5)
$H(S)$	Entropy of the set S (in bits)	Equation (5)
$\log_2$	Logarithm to base 2	Equation (5)
$G(S)$	Gini index of the set S	Equation (6)
A	Attribute (feature) used to split a node	Equation (7)
S_v	Subset of S in which attribute A takes the value v	Equation (7)
$\text{IG}(S, A)$	Information gain of attribute A on S	Equation (7)
$I(t)$	Impurity (Gini or entropy) of node t	Equation (8)
$\Delta\text{I}(t)$	Weighted impurity decrease obtained by splitting node t	Equation (8)
t_L, t_R	Left and right child nodes of node t	Equation (8)
N_t, N_L, N_R	Numbers of samples in node t and in its left and right children	Equation (8)
$\mathbf{x}_i$, $\mathbf{x}_{\text{nn}}$	Minority sample and one of its nearest minority neighbours	Equation (9)

Symbol	Meaning	First used in
$\mathbf{x}_{new}$	Synthetic sample generated by SMOTE	Equation (9)
δ	Random interpolation factor drawn from U(0, 1)	Equation (9)
$U(0, 1)$	Uniform distribution on the interval [0, 1]	Equation (9)
w_k	Weight of class k in cost-sensitive learning	Equation (10)
N	Number of training samples	Equation (10)
N_k	Number of training samples of class k	Equation (10)
F_1	F1-score, the harmonic mean of precision and recall	Equation (14)
M_k	Value of a metric computed for class k	Equation (16)
M_{macro}, $M_{weighted}$	Macro-averaged and weighted-averaged metric	Equation (16)
n_k	Number of test samples of class k	Equation (16)
Σ	Summation operator	Text

Chapter 1: Project Overview and Theoretical Foundations

This chapter first introduces the project: why it matters, the scientific problem it addresses, its objectives, the recent work it builds on, its practical applications and the challenges it faces. It then presents the theoretical foundations on which the implementation will rest, from cloud storage, data leakage and data loss prevention to the machine-learning methods, evaluation metrics and deployment path used in the project.

Introduction

This section explains the importance of the project, first for the general field of artificial intelligence and then for the specific field of data loss prevention in cloud computing.

Importance of the general field: Artificial Intelligence

Artificial Intelligence (AI) builds systems that perceive, reason, learn and act in ways that would normally require human intelligence. Its most productive sub-field, Machine Learning (ML), improves performance on a task by learning from data rather than by following hand-written rules [1]. ML has become a core tool in cybersecurity because it scales to millions of log events, generalises to attack patterns it has not seen in exactly the same form and, with models such as decision trees, produces rules that analysts can verify. Its main risk is that careless evaluation can make a weak model look strong [2], [3]; building AI systems that are both effective and trustworthy is the central challenge of the field.

Importance of the specific field: Data Loss Prevention in cloud computing

Organisations of every size have moved their data to the cloud. Gartner forecast worldwide end-user spending on public cloud services at about US$723.4 billion for 2025, a 21.5% increase over 2024 [4]. Cloud storage services make data reachable from anywhere, but they also concentrate sensitive records in places that are one misconfigured permission, one stolen password or one careless employee away from exposure. The Cloud Security Alliance lists misconfiguration, weaknesses in identity and access management, insecure interfaces and the absence of a coherent cloud security strategy among the top threats to cloud computing [5].

The financial impact is large and growing. IBM's 2026 *Cost of a Data Breach* study puts the global average cost of a breach at US$4.99 million, a 12% increase on the previous year and the highest value recorded [6]. Figure 1 shows how this figure has evolved. Verizon's 2026 Data Breach Investigations Report, which analysed more than 22,000 confirmed breaches, found that the human element was involved in 62% of them and that 48% involved a third party [7].

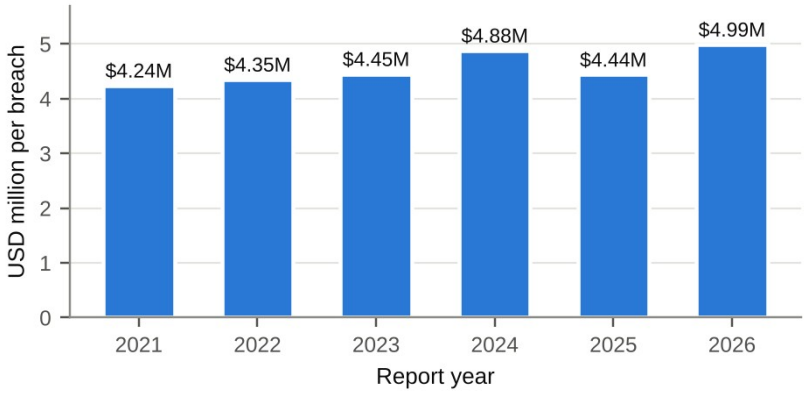


Figure 1: Global average cost of a data breach, 2021–2026

Values come from the yearly editions of IBM's Cost of a Data Breach report [6]. The 2025 dip was followed by a record high in 2026.

Data Loss Prevention is the discipline that tries to stop sensitive data from leaving the organisation without authorisation [8]. Classical DLP products rely on content inspection and hand-written policies. These work well for well-defined patterns, such as credit-card numbers in an outgoing e-mail, but they struggle with the behavioural side of leakage: a legitimate user who suddenly opens many confidential files and transfers them outside, or a stolen account that behaves differently from its owner. Detecting such behaviour from activity logs is a pattern-recognition problem, which is why ML-based DLP has become an active research area and a natural project for AI engineering students.

Scientific Problem

Cloud storage platforms record each user action as an event: who acted, how the user authenticated, what kind of operation was performed, how sensitive the data were, and whether confidential files were accessed or transferred. Most of these events are normal; a small fraction correspond to data leakage. The scientific problem of this project is therefore stated as follows:

How can individual cloud-storage activity events be classified automatically as *normal* or *data leakage* so that (1) a high proportion of real leakage events is detected, (2) the number of false alarms remains manageable for a security team, (3) the decisions can be explained to human analysts, and (4) the reported performance is obtained through an evaluation procedure free of methodological leakage, so that it reflects how the model would behave on new, unseen events?

The problem is harder than it first appears, for five reasons:

- **Leakage resembles legitimate work:** insiders and stolen accounts act within valid permissions, so the model must learn combinations of attributes.
- **The classes are imbalanced:** a model can look accurate by predicting "normal" for every event.
- **Errors have unequal costs:** a missed leak can cost millions, while too many false alarms cause alert fatigue.

- **Evaluation itself can leak:** preprocessing fitted before the split lets test information into training and inflates the scores [3].
- **The attributes are coarse:** nine categorical attributes allow at most 3,840 distinct events, so identical events can carry different labels, which caps what any model can achieve (see *Duplicate events and the performance ceiling*).

The reference article of this project [9] illustrates these difficulties: its tuned decision tree is barely more accurate than a model that always answers "normal" and detects only about a third of leakage events (Table 1). Improving this detection rate in a methodologically sound and interpretable way is the core of the present project.

Project Objectives

Summary of the reference article

The project is based on the article *Detecting Data Leakage in Cloud Storage Using Decision Tree Classification* by Harahap and Hasibuan, published in the *Journal of Information Systems and Informatics* in September 2025 [9]. The authors treat data-leakage detection as a binary classification task over a public Kaggle dataset of 49,498 cloud-storage activity events. Each event is described by nine categorical or binary attributes: the user's privilege level (`Authority`), three authentication flags (`Through_pwd`, `Through_pin`, `Through_MFA`), whether data were modified, whether confidential data were accessed or a confidential file was transferred, the type of operation and the sensitivity of the data. The target `Label` marks each event as normal or leakage.

After imputing missing values, removing outliers with the interquartile-range rule, one-hot encoding and min-max scaling, the authors split the data 80:20 with stratification, trained a CART decision tree in scikit-learn and tuned `max_depth`, `min_samples_split` and `criterion` with GridSearchCV. The best tree used entropy with a maximum depth of 3. Its feature importance is dominated by `Confidential_File_Transfer` (about 0.53) and `Confidential_Data_Access` (about 0.34), and only one path of the tree predicts leakage: a confidential transfer combined with confidential access by a user whose `Authority` code is at most 3. Table 1 compares the reported results with values that we recomputed from the published confusion matrix (TN = 8,310, FP = 1,081, FN = 2,885, TP = 1,329).

Table 1: Key results of the reference article and our recomputation

Metric	Reported in the article	Recomputed from the confusion matrix
Accuracy	0.71 (0.69 before tuning)	0.708
F1-score, macro average	0.60	0.604
Precision, leakage class	0.55	0.551
Recall, leakage class	not reported	**0.315**
Accuracy of always predicting "normal"	not reported	**0.690**

Bold values are our own calculations: the tuned model is only 1.8 percentage points more accurate than a trivial majority-class predictor and misses about two thirds of leakage events.

The article also leaves some methodological questions open. The test set behind its confusion matrix (13,605 records, about 27.5% of the data) does not match the stated 80:20 split; the preprocessing is not stated to have been fitted on the training data only, which risks preprocessing leakage [3]; and applying the IQR rule to binary attributes can delete every event in which a rare flag equals 1 (see *Outliers and the interquartile-range rule*). The authors themselves note that the data are synthetic.

What this project aims to implement

The project takes the reference article as its baseline and builds a leakage-aware detector that runs as a small cloud-style service (Figure 2). Its objectives are:

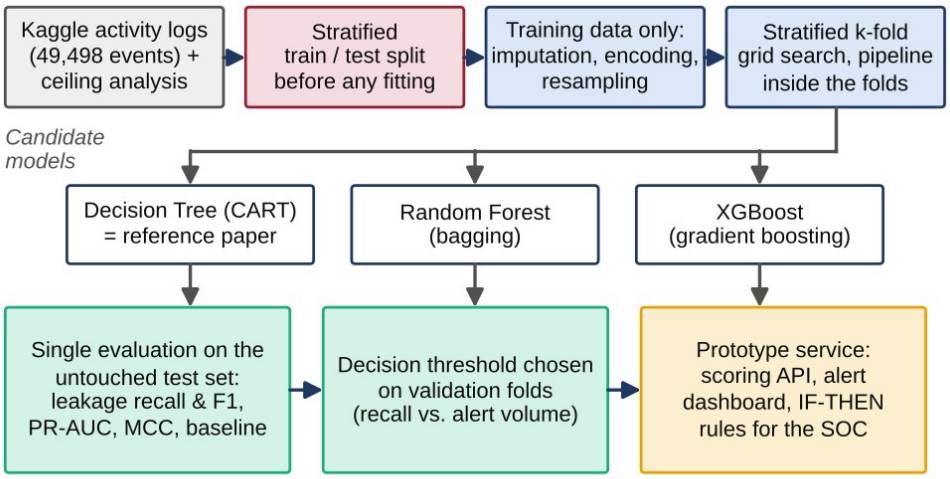


Figure 2: Proposed leakage-aware detection framework

Grey: ceiling analysis; red: split before any fitting; blue: fitting on training data only; green: single test evaluation; yellow: prototype service.

1. **Reproduce the baseline:** the reference CART tree on the same dataset.
2. **Measure the ceiling first:** compute the best score the attributes allow (see *Duplicate events and the performance ceiling*).
3. **Build a leakage-safe pipeline:** fit every step on the training folds only [10].
4. **Handle class imbalance:** class weights, SMOTE variants and threshold tuning.
5. **Compare models:** decision tree, Random Forest and XGBoost under identical conditions.
6. **Use suitable metrics:** leakage-class recall, precision, F1, PR-AUC and MCC.
7. **Keep the result explainable:** IF-THEN rules and permutation importance.
8. **Deploy a prototype:** scoring API and alert dashboard on audit-log events (see *From Model to Cloud Service*).

Related Works

Three recent studies, published in 2025 and 2026, were selected in addition to the reference article. Each covers a different side of the project: how to evaluate a model without

methodological leakage, how supervised learning predicts data exfiltration, and how anomaly detection and a SIEM platform detect insider threats in cloud environments. Table 2 at the end of the section summarises them.

Study 1: Data leakage risks in machine learning (Apicella et al., 2025)

Apicella, Isgrò and Prevete [3] study *data leakage* in its machine-learning sense: forbidden information that reaches the training process and inflates the measured performance. They argue that easy-to-use ML libraries encourage a "push the button" approach in which crucial pipeline steps are applied without understanding their consequences. The paper offers a formal definition of leakage relative to a standard ML pipeline and a taxonomy of three families. *Data-induced leakage* arises while the dataset is built (data collecting, label and synthesis leakage, the latter including oversampling such as SMOTE before the split). *Preprocessing-related leakage* arises when normalisation, cleaning, imputation or feature engineering are fitted before the split. *Split-related leakage* arises from duplicates or mismatched distributions across training and test sets. The authors also show that leakage depends on the learning paradigm, inductive or transductive, and on whether transfer learning is used, and they close with a table of safe practices.

Relation to the project. Although the word "leakage" refers here to an evaluation flaw rather than a security incident, this study is directly relevant. The reference article applies imputation, IQR-based outlier removal and min-max scaling without stating that they were fitted on training data only, which matches the preprocessing-related category. The taxonomy gives the project a checklist for designing its pipeline (Figure 10) and for judging the credibility of results reported in the literature.

Study 2: ARKAIV, predicting data exfiltration (Hakim et al., 2025)

Hakim et al. [11] propose ARKAIV, a supervised method that predicts whether an attack will end in data exfiltration from the sequence of attacker tactics observed in event logs. They extract ten MITRE ATT&CK tactics from published threat reports to build a dataset in which exfiltration is the target; the cleaned data are strongly imbalanced, with a 93% to 7% class ratio. The authors compare random undersampling, SMOTE and the hybrid SMOTE+ENN, which gave the best results; select features with mutual information, Pearson correlation, chi-square, random-forest importance and recursive feature elimination, retaining four tactics (Discovery, Lateral Movement, Collection and Command and Control); and benchmark Random Forest, SVM, logistic regression, XGBoost and naive Bayes. Random Forest performed best, with mean cross-validated accuracies above 0.99. In a case study, Windows event IDs from three replicated incidents were mapped to ATT&CK tactics through a lookup table, and the model correctly separated the two exfiltration incidents from the ransomware incident.

Relation to the project. ARKAIV addresses the same security goal, predicting exfiltration, with the same family of tree-based models that this project will compare. It also shows the value of handling imbalance explicitly. At the same time, it illustrates the methodological risk raised by Study 1: the balanced dataset produced by SMOTE+ENN is the one that was later divided into cross-validation folds, so synthetic samples derived from

the same original records can fall on both sides of a split. Apicella et al. classify this as synthesis leakage, and it may explain part of the near-perfect accuracy. Our project will therefore resample inside the training folds only.

Study 3: Insider-threat detection in cloud environments (Komka et al., 2026)

Komka et al. [12] present an end-to-end framework for detecting insider threats in cloud-based enterprises. Using the CERT Insider Threat Dataset v6.2, they encode user, device, activity type, hour and weekday, fit the min-max scaler on normal records only, and train two unsupervised detectors, Isolation Forest and One-Class SVM, on 531,921 normal records. To test cloud-style attacks, they inject four scenarios modelled on AWS audit events: bulk `GetObject` access at 2 a.m. (exfiltration staging), repeated `AssumeRole` calls (privilege escalation), a burst of `DeleteObject` events and abnormal night-time logins. On a 909-record evaluation set, Isolation Forest reached a weighted F1-score of 0.92 (insider-class precision 0.89, recall 0.87), while One-Class SVM reached 0.81 with perfect insider recall but a precision of 0.62. Predictions were visualised in Splunk Cloud dashboards, and a live integration through Splunk's HTTP Event Collector is proposed.

Relation to the project. This study works in the same environment, cloud storage, and against the same threat, insiders who misuse legitimate access. It contributes three ideas: an unsupervised alternative when labels are scarce; fitting preprocessing on training data only to avoid leakage; and a practical route from model output to analyst dashboards. Its authors acknowledge that the injected events are structurally obvious and that the scores are therefore an optimistic upper bound, a limitation that reinforces our emphasis on careful evaluation.

Summary of the related works

Table 2: Summary of the selected related works

Article title	Author	Publisher	Year	Relation with the project
Don't push the button! Exploring data leakage risks in machine learning and transfer learning	A. Apicella, F. Isgrò, R. Prevete	Springer Nature (Artificial Intelligence Review)	2025	Taxonomy and safe practices for avoiding methodological leakage; used to design and audit the evaluation pipeline.
ARKAIV: Predicting data exfiltration using supervised machine learning based on tactics mapping from threat reports and event logs	A. R. Hakim, K. Ramli, M. Salman, B. Pranggono, E. R. Agustina	IEEE (IEEE Access)	2025	Same goal (exfiltration prediction) with tree ensembles and SMOTE+ENN; motivates imbalance handling and the DT vs. RF vs. XGBoost comparison.
Proactive insider threat detection in cloud environments using behavior-aware machine learning models	M. C. Komka, A. K. Ademuwagun, S. M. Abdullahi, T. M. Mashkur, F. D. Ifeanyi	AIR NATURE (NJESTI)	2026	Same environment and threat (cloud insiders); unsupervised baseline, training-only scaling and SIEM integration of alerts.

Publication details are listed in the References. The reference article itself [9] is excluded from this table, as required.

Practical Applications of the Project

A model that labels cloud-storage events as normal or leakage, and explains its decisions, can serve several practical purposes:

- **Real-time risk scoring in DLP products.** Each event from a storage service such as Amazon S3, Google Drive or Microsoft OneDrive receives a leakage score. Events above a threshold can be blocked, quarantined or held for approval, which adds a behavioural layer to content-based DLP policies. The project's prototype demonstrates this path with a scoring API and an alert dashboard.
- **Alert prioritisation in a Security Operations Centre (SOC).** Scores and rules can be forwarded to a SIEM platform, as demonstrated by Komka et al. [12], so that analysts start with the events most likely to be leakage and receive the rule that triggered each alert.
- **Insider-threat programmes and user behaviour analytics.** Repeated high scores for one account can trigger a review, a conversation with the employee's manager or a temporary reduction of privileges.
- **Adaptive authentication.** When an event is risky, the system can demand step-up authentication, for example multi-factor authentication, before allowing a confidential transfer.
- **Policy design and auditing.** Rules extracted from the tree (for example, "confidential transfer and confidential access by a user with authority code ≤ 3") can be reviewed by security officers and turned into explicit DLP policies or audit checks.
- **Compliance and forensics.** Regulations such as the GDPR require organisations to protect personal data and to notify breaches. A log of scored events supports investigations and helps demonstrate due diligence.
- **Low-cost protection for small organisations.** Decision trees and tree ensembles train quickly on ordinary hardware, which suits universities, clinics and small companies that cannot afford commercial UEBA platforms.

Scientific Challenges

The project faces several scientific and engineering challenges. Table 3 lists each challenge together with the way the project intends to address it.

Table 3: Scientific challenges and the planned response

Challenge	Why it matters	Planned response
Class imbalance	Leakage events form only about 30% of the data; models drift towards the majority class and accuracy becomes misleading.	Class weights, SMOTE and SMOTE+ENN on training folds only; threshold tuning; metrics focused on the minority class.
Methodological leakage	Fitting preprocessing or resampling on the full dataset inflates test scores [3].	Split first; wrap every fitted step in a pipeline evaluated with stratified cross-validation.
Coarse attributes and a performance ceiling	Only 3,840 distinct events are possible, so identical events can carry different labels; no model can exceed the resulting ceiling.	Compute the ceiling before modelling and report all results relative to it, even if the achievable gain is small.
Synthetic, limited data	The Kaggle data are synthetic and lack time, volume and location context; results may not transfer to real logs.	Treat results as a proof of concept; design the pipeline so that richer features can be added later.
Recall versus false alarms	Missed leaks are costly, but excessive alerts cause analyst fatigue.	Use precision-recall curves and choose the operating threshold from validation data.
Interpretability versus accuracy	Ensembles are usually more accurate but harder to explain than one tree.	Compare both; extract rules from the tree and use permutation importance for the ensembles.
From offline model to cloud service	Real audit logs differ in format from the Kaggle table, and alerts must arrive in time.	Map audit-log fields to the attributes; build a scoring API and dashboard; replay test events and measure latency.
Evasion by informed insiders	Someone who knows the extracted rules can avoid them, e.g. by relabelling files or spreading a theft over many small events.	Simulate evasive variants of leakage events and report how detection degrades; discuss session-level features.
Concept drift	User behaviour and attack techniques change over time.	Monitor score distributions and alert rates; retrain periodically.
Privacy and ethics	Monitoring employees' activity raises privacy concerns.	Use only the attributes needed, aggregate where possible, and keep a human in the loop for decisions.

The first three challenges are the ones that most limited the reference article and are therefore given the highest priority in this project.

Cloud Computing and Cloud Storage

Definition and essential characteristics

The U.S. National Institute of Standards and Technology (NIST) defines cloud computing as a model for enabling convenient, on-demand network access to a shared pool of configurable computing resources that can be provisioned and released rapidly with minimal management effort [13]. Its five essential characteristics are *on-demand self-service*, *broad network access*, *resource pooling*, *rapid elasticity* and *measured service*. Each of them widens the attack surface: broad network access means that a stolen

password works from anywhere, resource pooling makes isolation between tenants depend on correct configuration, and self-service lets a single administrator make a storage bucket public in seconds.

Service models and the shared-responsibility principle

NIST distinguishes three service models [13]. In *Infrastructure as a Service* (IaaS) the provider supplies virtual machines, storage and networks, while the customer manages the operating system and everything above it. In *Platform as a Service* (PaaS) the provider also manages the operating system and runtime. In *Software as a Service* (SaaS) the customer simply uses a finished application, such as a file-sharing service. Figure 3 shows how responsibility moves from the customer to the provider across the models. Typical leakage risks differ accordingly: misconfigured buckets and exposed snapshots in IaaS, over-privileged service accounts in PaaS, and public sharing links, compromised accounts and insider downloads in SaaS.

This division is known as the *shared-responsibility model*: the provider secures the cloud itself, while the customer remains responsible for security *in* the cloud: its data, identities, access rights and configuration. Most cloud data leaks happen on the customer side of this line, which is why monitoring user activity is the customer's task. Cloud services are also deployed as public, private, community or hybrid clouds [13]; organisations that combine several of them multiply the places in which sensitive data may reside.

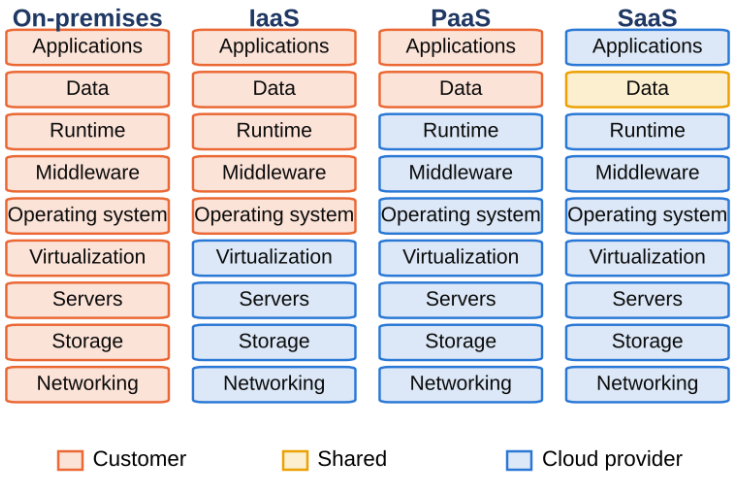


Figure 3: Division of responsibility across cloud service models

The higher the service model, the more layers the provider manages. Data remain at least a shared responsibility in every model, because only the customer decides who may access them.

Cloud storage and its threat landscape

Cloud storage services store data as *objects* (files plus metadata) inside containers, often called buckets, and expose them through web interfaces and application programming interfaces (APIs). Typical operations are reading (e.g. `GetObject`), writing (`PutObject`), copying, sharing and deleting objects. Providers can record each operation in audit logs together with the identity of the caller, the authentication context and the time (see *From Model to Cloud Service*). These logs are the raw material for behaviour-based detection;

the dataset of the reference article [9] is a simplified, tabular version of such a log, with one row per event.

The Cloud Security Alliance places misconfiguration, identity and access management weaknesses, insecure interfaces and an inadequate cloud security strategy at the top of its 2024 list of cloud threats, alongside accidental data disclosure and limited visibility [5]. Identity weaknesses and limited visibility cannot be removed by configuration alone; they require continuous monitoring of how identities use the data.

Information Security and Data Leakage

Confidentiality and data leakage

Information security is usually described through the CIA triad: *confidentiality* (data are disclosed only to authorised parties), *integrity* (data are not modified without authorisation) and *availability* (data are accessible when needed). Data leakage is first of all a violation of confidentiality, although leakage incidents are often combined with deletion or tampering, as the `Operation` and `Data_Modification` attributes of the dataset suggest. Several related terms should be distinguished:

- **Data leakage** is the unauthorised transmission of data from inside an organisation to an external destination or recipient, whether intentional or accidental [8].
- **Data loss** also covers data destroyed or made unavailable; DLP products address both, and the acronym is read as *data leakage prevention* or *data loss prevention* [8].
- **Data exfiltration** is the deliberate, often attacker-driven removal of data; in MITRE ATT&CK it is the tactic in which an adversary steals data it has already collected [11].

The word *leakage* also has an unrelated meaning in machine learning: information from the evaluation data that contaminates training [3]. This report writes *data leakage* or *leakage event* for the security incident and *methodological leakage* for the evaluation flaw discussed later in this chapter.

Sources of data leakage and the insider threat

Leakage can originate from several kinds of actors and causes, summarised in Table 4. Of particular importance is the *insider threat*: a current or former employee, contractor or business partner who has or had authorised access and misuses it, intentionally or not [14]. Insiders are hard to detect because their actions use valid credentials and fall within their permissions.

Table 4: Main sources of data leakage in cloud storage

Source	Description	Cloud-storage example	Observable indicators
Malicious insider	Authorised user who deliberately steals data	Employee copies confidential designs before resigning	Access to confidential data, unusual transfers, unusual operations
Negligent insider	Authorised user who exposes data by mistake	Staff member shares a folder through a public link	Confidential transfer without malicious intent; sharing events
Compromised account	External attacker using stolen or guessed credentials	Phished password used to download a bucket	Weak authentication (password only), atypical behaviour for the role
Misconfiguration	Incorrect settings expose data to everyone	Storage bucket left publicly readable	Configuration changes; access by unknown identities
Third party	Partner, vendor or integrated application with excessive access	Analytics plug-in with read access to all files	Service accounts reading large volumes of sensitive data

Indicators in the last column correspond closely to the attributes of the project dataset (authentication method, confidential access, confidential transfer, operation type and user authority).

Anatomy of a leakage event in cloud storage

Most leakage incidents follow a common path, illustrated in Figure 4. The user authenticates, accesses sensitive objects, possibly collects or modifies them, and finally transfers them outside the organisation. Each stage leaves traces in the audit log and corresponds to one or more attributes of the project dataset. A behaviour-based detector looks at these traces jointly: an isolated confidential access is routine, but confidential access followed by a confidential transfer by a user with an unusual role is much more suspicious. Decision trees capture such conjunctions naturally, which is one reason they are used in this project.

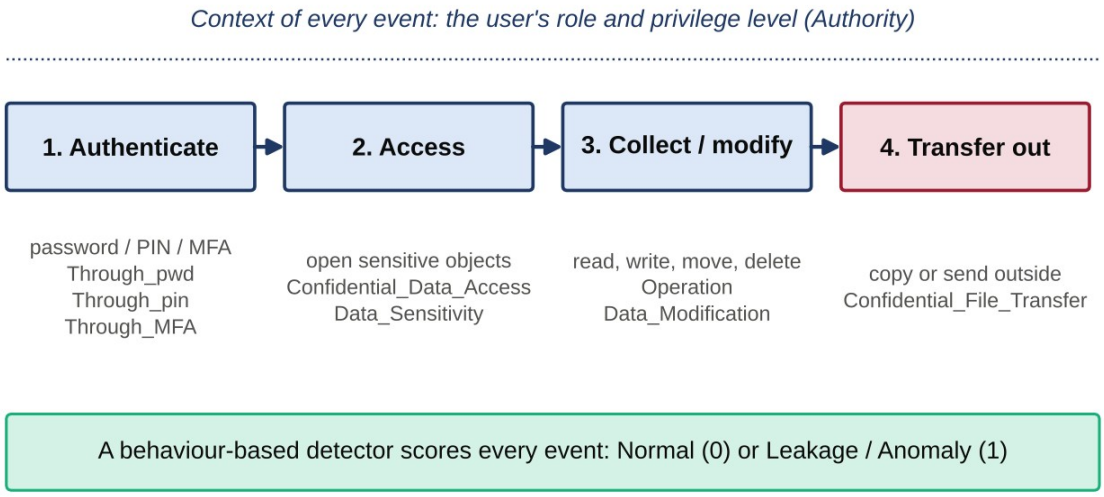


Figure 4: Stages of a leakage event in cloud storage and the related dataset attributes

Each stage leaves evidence in the activity log. The attribute names under each stage are those of the dataset used by the reference article; Authority provides context for all stages.

Threat model of the project

A threat model states what a system protects, against whom, under which assumptions, and what it does not attempt to detect. The *asset* in this project is the set of confidential objects held in the organisation's cloud storage. The detector observes only authenticated activity recorded in the audit log, one event at a time, and assumes that the log is complete and cannot be altered by the adversary. Table 5 lists the adversaries considered and whether the detector can, in principle, see them.

Table 5: Threat model: adversaries and coverage by the detector

Adversary	Goal and capability	Trace in the audit log	Covered?
Malicious insider	Steals data using a legitimate role and valid credentials	Confidential access followed by transfer; unusual operations for the role	Yes (main target)
Negligent insider	No malicious goal; exposes data by mistake	Confidential transfer or sharing, often with weak authentication	Yes (flagged for review)
Compromised account	External attacker using stolen credentials	Valid login, often password only; actions atypical for the role	Partly
Log tampering	Disables or edits audit logs before acting	Gap or change in the log stream	No (needs a log-integrity rule)
Out-of-band leakage	Photographs, printing or USB copies on an endpoint	Not visible in cloud logs	No (endpoint DLP)

The detector targets the first three adversaries; the last two require other security controls.

To speak the common language of security teams, the targeted behaviours can be mapped to MITRE ATT&CK, the public knowledge base of adversary tactics and techniques [15]. Table 6 relates each relevant technique to the dataset attributes that provide evidence for it. The mapping shows that the dataset covers the *Collection* and *Exfiltration* stages well, the misuse of valid accounts only partially, and tampering with logs not at all.

Table 6: Mapping of the targeted behaviours to MITRE ATT&CK techniques

Tactic	Technique (ATT&CK ID)	Evidence in the dataset attributes
Initial Access, Persistence	Valid Accounts: Cloud Accounts (T1078.004)	`Through_pwd`, `Through_pin`, `Through_MFA`, `Authority`
Collection	Data from Cloud Storage (T1530)	`Confidential_Data_Access`, `Operation` = read, `Data_Sensitivity`
Exfiltration	Transfer Data to Cloud Account (T1537); Exfiltration to Cloud Storage (T1567.002)	`Confidential_File_Transfer`, `Operation` = move
Impact	Data Destruction (T1485)	`Operation` = delete, `Data_Modification`
Defense Impairment	Disable or Modify Cloud Log (T1685.002)	Not observable in the dataset; needs a separate rule

Names and identifiers follow ATT&CK version 19. Valid Accounts also belongs to the Stealth and Privilege Escalation tactics.

An interpretable detector raises one more security question: an informed insider who learns the extracted rules may try to *evade* them, for example by first copying confidential files to an unlabelled location so that the later transfer no longer carries the confidential flag, or by

spreading a theft over many small, normal-looking events. Single-event classification cannot see such patterns. The project will therefore test simple evasive variants of leakage events and report how detection degrades, while label persistence and session-level features remain natural extensions for future work.

Data Loss Prevention

Concept and the three states of data

Data Loss Prevention (DLP) refers to the technologies, policies and processes that identify sensitive data, monitor how they are used and moved, and prevent their unauthorised disclosure [8]. DLP distinguishes three states of data, shown in Figure 5: *data at rest* in databases, file shares or cloud buckets; *data in motion* across networks; and *data in use* on an endpoint or inside an application. The project focuses on the boundary between rest and motion: it analyses access and transfer events recorded by a cloud storage service.

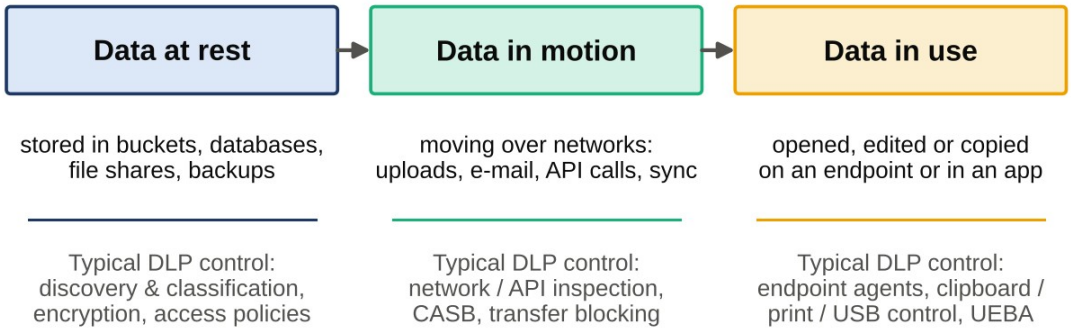


Figure 5: The three states of data and typical DLP controls for each

Cloud-storage audit logs mainly describe the transition from data at rest to data in motion, which is where the project's detector operates.

Functional architecture of a DLP system

Figure 6 shows the main functions of a DLP system. *Discovery* finds sensitive data; *classification* labels them by sensitivity (the `Data_Sensitivity` attribute is an example of such a label); *monitoring* collects access and transfer events; *detection* decides whether an event violates policy; and *response* blocks, encrypts or quarantines the data or raises an alert. Alerts are forwarded to a Security Information and Event Management (SIEM) system used by the Security Operations Centre (SOC); in the cloud these functions are often delivered by a Cloud Access Security Broker (CASB). The model developed in this project belongs to the *detection* function.

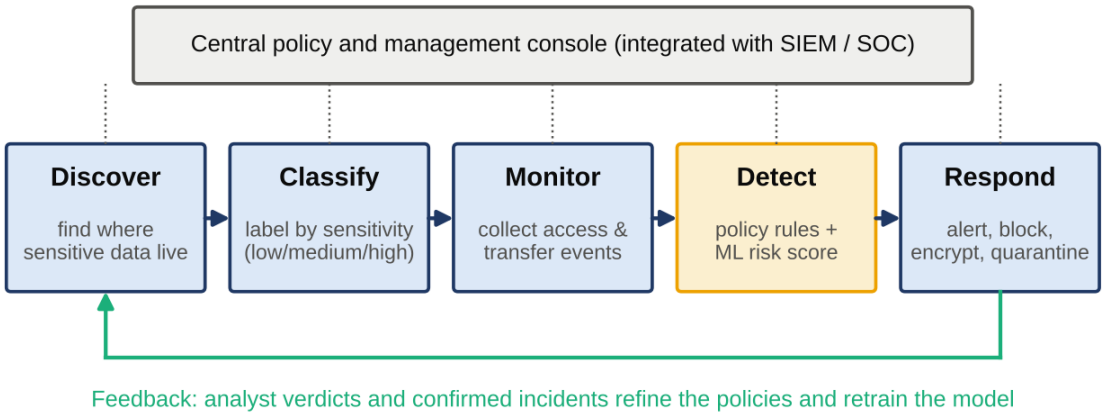


Figure 6: Functional architecture of a data loss prevention system

The project contributes to the Detect block (yellow) by adding a learned risk score to rule-based policies. Analyst feedback closes the loop for retraining.

Rule-based and learning-based detection

DLP detection techniques are *content-based* (regular expressions, keyword lists, document fingerprints), *context-based* (who acts, from where, on which file, with which application) or *behaviour-based*, also called user and entity behaviour analytics (UEBA), which model normal behaviour and flag deviations [8]. Table 7 compares the traditional rule-based approach with the learning-based approach adopted here.

Table 7: Rule-based versus machine-learning-based leakage detection

Aspect	Rule-based DLP	Machine-learning-based detection
Knowledge source	Experts write explicit policies	Patterns are learned from labelled or normal data
Novel behaviour	Missed unless a rule anticipates it	Can generalise to related, previously unseen patterns
Insider misuse within permissions	Hard to express as fixed rules	Detected as unusual combinations of attributes
False positives	Often high for broad rules	Adjustable through the decision threshold
Explainability	Very high: the rule is the explanation	Depends on the model; high for decision trees
Maintenance	Rules must be updated by hand	Models must be retrained and monitored for drift

The two approaches are complementary: rules handle well-defined content patterns, while learned models add a behavioural layer. Decision trees are attractive because they produce rules that look like DLP policies.

Identity, Authentication and Access Control

Authentication verifies that a user is who they claim to be. NIST groups authenticators into three factors : something you *know* (a password or PIN), something you *have* (a phone or hardware token) and something you *are* (a biometric characteristic). Multi-factor authentication (MFA) combines at least two different factors, so that a stolen password

alone is not enough to log in. Table 8 relates these concepts to the authentication attributes of the dataset.

Table 8: Authentication factors and the corresponding dataset attributes

Factor	Examples	Dataset attribute	Relevance to leakage
Knowledge	Password, passphrase	`Through_pwd`	Passwords can be phished, guessed or reused; a password-only login is weaker evidence of identity
Knowledge	Personal identification number (PIN)	`Through_pin`	Short PINs are easy to observe or guess; often used on devices
Two or more factors	Password + one-time code, password + security key	`Through_MFA`	Greatly reduces account takeover but does not stop a legitimate insider

Strong authentication protects against external attackers using stolen credentials, but not against misuse by the legitimate account owner. This is why behavioural attributes are also needed.

Once identity is established, access control decides what the user may do. In *role-based access control* (RBAC) permissions are attached to roles such as staff, manager or intern; in *attribute-based access control* (ABAC) decisions also depend on attributes of the resource and the environment. The `Authority` attribute of the dataset reflects an RBAC model. Even a least-privilege RBAC policy cannot tell whether a permitted action serves a legitimate purpose; activity monitoring complements it by asking whether *this particular use* of a permission is consistent with normal behaviour.

Machine Learning for Leakage Detection

Learning paradigms

A program learns from experience E with respect to a task T and a performance measure P if its performance at T, measured by P, improves with E [1]. Here T is the classification of cloud-storage events, E is a set of labelled historical events and P is, for example, the recall or F1-score of the leakage class. *Supervised* learning, used by the reference article and by this project, needs a label for every training event. *Unsupervised* anomaly detection learns only what normal behaviour looks like and flags deviations; Isolation Forest and the One-Class SVM, used by Komka et al. [12], belong to this family. They need no labels and can flag new behaviour, but they also flag unusual yet harmless events, so in this project they serve only as reference points.

Formulation of the detection task

Let the dataset be $D = \{(\mathbf{x}_i, y_i)\}$, $i = 1, \ldots, n$, where $\mathbf{x}_i$ is the feature vector of event i (its authority, authentication flags, operation and so on) and $y_i \in \{0, 1\}$ is its label, with 1 denoting leakage. A probabilistic classifier produces a score $s(\mathbf{x}) \in [0, 1]$ that estimates the probability of leakage, and a decision threshold θ converts it into a prediction, as in Equation (1). The default threshold is 0.5, but in security applications θ is a tunable parameter: lowering it detects more leakage at the price of more false alarms. The model is built with the usual supervised sequence (split, preprocess, train, tune, evaluate), in which

the position of the split is the most important design decision (see *Methodological Data Leakage in Machine Learning*).

$$\hat{y} = \begin{cases} 1 & \text{if } s(\mathbf{x}) \geq \theta \\ 0 & \text{otherwise} \end{cases} \tag{1}$$

Data Preprocessing and the Limits of the Dataset

Preprocessing converts raw data into the form an algorithm expects. Every step that *learns* something from the data, such as a median, a category frequency or a minimum and maximum, must learn it from the training data only. This section focuses on the steps used in the reference article and on the properties of its dataset that limit what any model can achieve.

Missing values

Missing values are either deleted or imputed, typically with the mode for categorical attributes and the median for numerical ones, as in the reference article [9]. Computing these values on the full dataset is a mild but real form of preprocessing leakage [3]. In security data the absence of a value can itself be informative, so a binary "was missing" indicator is sometimes better than imputation.

Outliers and the interquartile-range rule

Tukey's rule flags values outside the fences of Equation (2), where Q_1 and Q_3 are the 25th and 75th percentiles. The rule is designed for continuous variables, and the reference article applied it to a dataset whose attributes are all binary or categorical. For a binary attribute in which fewer than 25% of the values equal 1, both quartiles equal 0, the IQR is 0, and *every* event with the flag set to 1 becomes an "outlier". Removing them would delete, for instance, all confidential transfers, the very events that signal leakage. The project will therefore inspect outliers rather than remove them, and only on training data.

$$\mathrm{IQR} = Q_3 - Q_1, \qquad [\,Q_1 - 1.5\,\mathrm{IQR},\; Q_3 + 1.5\,\mathrm{IQR}\,] \tag{2}$$

Encoding and scaling

Nominal attributes such as `Operation` (delete, read, move, write) are usually one-hot encoded, while *ordinal* attributes such as `Data_Sensitivity` (low, medium, high) can be encoded as order-preserving integers. The tree published in the reference article splits on `Authority <= 3.5`, so `Authority` was in fact used as an integer code, although the text states that it was one-hot encoded [9]. Because the codes do not follow a strict privilege order (Intern is coded 5 and Expert Staff 4), the project will compare both representations.

Min-max normalisation (Equation (3)) maps each value to [0, 1] using the minimum and maximum of the training data. It matters for distance-based methods, including SMOTE (see *Learning from Imbalanced Data*), but has no effect on decision trees or tree ensembles: a monotonic transformation does not change the ordering of a feature's values

and therefore yields the same splits. The min-max step of the reference article could be omitted without changing its tree.

$$x' = \frac{x - x_{\min}}{x_{\max} - x_{\min}} \tag{3}$$

Duplicate events and the performance ceiling

The most important property of the dataset is how coarse it is. With five authority levels, six binary flags, four operation types and three sensitivity levels, at most $5 \times 2^6 \times 4 \times 3 = 3{,}840$ distinct events can exist, while the dataset contains 49,498 rows, about 13 per combination on average. Every classifier, however complex, must give the same answer to identical inputs. If a combination v occurs n_v times and m_v of those events are labelled as leakage, the best possible decision is the majority label of v, and the highest accuracy that any model can reach is the Bayes-optimal accuracy of Equation (4), where V is the set of observed combinations [16].

$$\mathrm{Acc}^* = \frac{1}{n} \sum_{v \in V} \max\left(m_v,\ n_v - m_v\right) \tag{4}$$

Table 9 illustrates the idea with four invented combinations. Even a perfect model reaches only 76% accuracy and 30% recall when it predicts the majority label; flagging combination B as well raises recall to 57% but lowers precision to 53%. The ranking of combinations by their leakage rate m_v/n_v also gives the best achievable precision-recall curve.

Table 9: Illustrative calculation of the performance ceiling for four attribute combinations

Combination	Events n_v	Leakage events m_v	Leakage rate	Majority label	Correct predictions
A (transfer = 1, access = 1, …)	120	90	0.75	Leakage	90
B (transfer = 1, access = 0, …)	200	80	0.40	Normal	120
C (transfer = 0, access = 0, …)	500	60	0.12	Normal	440
D (transfer = 0, access = 1, …)	180	70	0.39	Normal	110
Total	**1,000**	**300**	–	–	**760 (Acc* = 0.76)**

Invented numbers for illustration only. The project will compute the real ceiling from the Kaggle data before training any model.

The reference article's tuned tree reaches 70.8% accuracy and 31.5% recall, which suggests that the real dataset overlaps heavily. The project will therefore measure the number of distinct combinations, the share of events in combinations with conflicting labels and the resulting ceiling *before* modelling, and will report every result relative to it. If the ceiling is low, the honest conclusion is that the attributes, not the algorithm, limit detection. Coarseness also affects evaluation: almost every test event has identical copies in the

training set, so stratified splitting must be complemented by reporting this overlap (see *Methodological Data Leakage in Machine Learning*).

Decision Trees

Structure and rules

A decision tree predicts a class by asking a sequence of questions about the features [17]. Each *internal node* tests one feature, each *branch* is an outcome of the test, and each *leaf* assigns a class and a probability equal to the share of training samples of that class in the leaf. Figure 7 shows a small tree inspired by the one reported in the reference article. Every path from the root to a leaf is an IF-THEN rule, for example: IF `Confidential_File_Transfer` = 1 AND `Confidential_Data_Access` = 1 AND `Authority` ≤ 3 THEN *Leakage*. Such rules can be reviewed by security officers and turned into DLP policies.

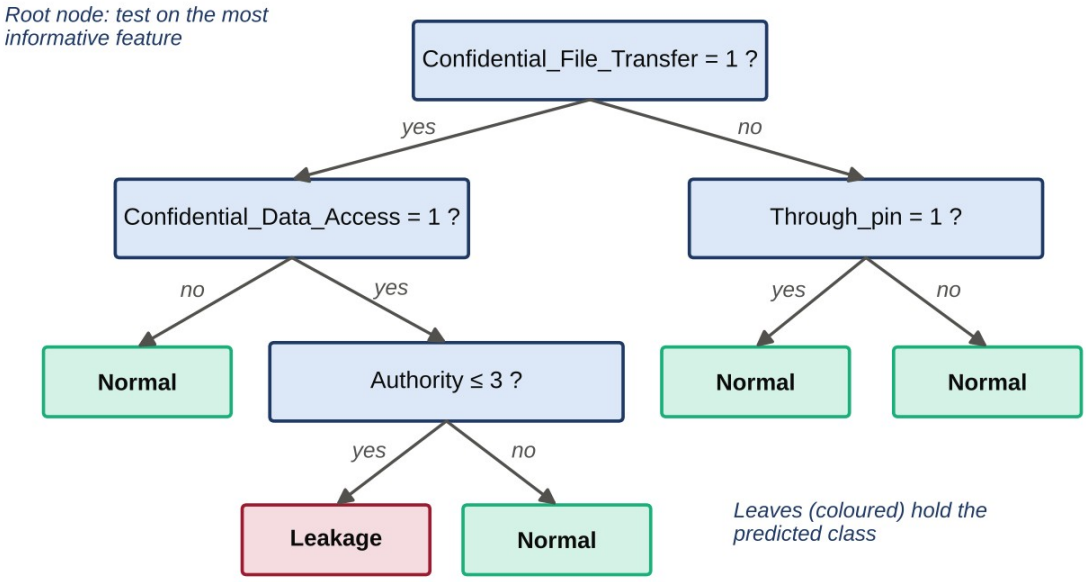


Figure 7: Example of a decision tree for leakage detection

Simplified illustration based on the attributes and the single leakage path of the tree reported in [9]. Blue nodes are tests; green and red leaves are the predicted classes.

Choosing splits: entropy, Gini and information gain

A tree is grown by *recursive partitioning*: at each node the algorithm chooses the test that makes the children as *pure* as possible. Let S be the samples at a node, K the number of classes and p_k the share of class k in S. Impurity is measured by entropy (Equation (5)) or by the Gini index (Equation (6)); both are zero for a pure node and maximal for an even mix. ID3 and C4.5 select the attribute A with the largest *information gain* (Equation (7)), C4.5 normalising it into a *gain ratio*. CART, implemented in scikit-learn [10], makes binary splits that maximise the weighted impurity decrease of Equation (8) [17].

$$H(S) = -\sum_{k=1}^{K} p_k \log_2 p_k \tag{5}$$

$$G(S) = 1 - \sum_{k=1}^{K} p_k^2 \tag{6}$$

$$IG(S,A) = H(S) - \sum_{v \in \text{Values}(A)} \frac{|S_v|}{|S|} H(S_v) \tag{7}$$

$$\Delta I(t) = I(t) - \frac{N_L}{N_t} I(t_L) - \frac{N_R}{N_t} I(t_R) \tag{8}$$

Worked example. Suppose a node contains 1,000 events, 300 of which are leakage. Splitting on `Confidential_File_Transfer` sends 400 events to the left child (transfer = 1), of which 220 are leakage, and 600 events to the right child, of which 80 are leakage. Table 10 applies Equations (5) to (8) to this split.

Table 10: Worked example of split evaluation with entropy and the Gini index

Quantity	Parent (1,000)	Left child (400)	Right child (600)	Result of the split
Leakage proportion p	0.300	0.550	0.133	–
Entropy H	0.881	0.993	0.567	Weighted H = 0.737
Information gain IG	–	–	–	0.881 − 0.737 = **0.144** bits
Gini index G	0.420	0.495	0.231	Weighted G = 0.337
Gini decrease ΔI	–	–	–	0.420 − 0.337 = **0.083**

The numbers are illustrative. The algorithm computes the same quantities for every candidate split of every feature and keeps the split with the largest gain.

Overfitting, pruning and hyper-parameters

A tree grown until every leaf is pure memorises the training data, including its noise, and generalises poorly [16]. Growth is limited by the hyper-parameters in Table 11 or by *cost-complexity pruning*, which removes branches whose contribution does not justify their size [17]. With only 3,840 possible inputs, a deep tree can approach a lookup table of per-combination majority labels, which is exactly the Bayes-optimal classifier described above; the practical question is how much of that ceiling a small, readable tree retains.

Table 11: Main hyper-parameters of scikit-learn's DecisionTreeClassifier

Hyper-parameter	Meaning	Effect of increasing it
`criterion`	Impurity measure: "gini", "entropy" or "log_loss"	–
`max_depth`	Maximum depth of the tree	More complex tree, higher risk of overfitting
`min_samples_split`	Minimum samples required to split a node	Fewer splits, simpler tree
`min_samples_leaf`	Minimum samples required in each leaf	Smoother probabilities, simpler tree
`class_weight`	Weights of the classes in the impurity computation	"balanced" gives the minority class more influence
`ccp_alpha`	Cost-complexity pruning parameter	Stronger pruning, smaller tree

The reference article tuned only criterion, max_depth and min_samples_split. The project will also tune min_samples_leaf, class_weight and ccp_alpha.

Feature importance

The *mean decrease in impurity* (MDI) of a feature sums the weighted impurity decreases of all nodes that split on it; the feature-importance chart of the reference article is of this type. MDI is computed on training data and is biased towards features with many split points, so the project will also report *permutation importance*, the drop in a held-out metric when one feature is shuffled.

Tree Ensembles

Single trees are unstable: a small change in the data can produce a different tree. Ensembles reduce this variance. *Random Forest* [18] trains many trees on bootstrap samples, considering a random subset of features at each split, and combines them by voting; it was the best model in the exfiltration study of Hakim et al. [11]. *Gradient boosting*, efficiently implemented in XGBoost [19], adds trees one after another, each fitted to the errors of the current ensemble, with regularisation against overly complex trees and a `scale_pos_weight` option that reweights the minority class. Both usually outperform a single tree, but replace one readable rule set with hundreds of trees, so the project will explain them with permutation importance and SHAP values. On a dataset as coarse as this one, however, ensembles cannot exceed the performance ceiling, and the gain over a well-tuned tree may be small.

Learning from Imbalanced Data

A dataset is imbalanced when one class is much rarer than another. In the reference dataset roughly 70% of events are normal, an imbalance ratio of about 2.3; in real cloud logs leakage is far rarer. Learning algorithms that minimise overall error then favour the majority class, and accuracy becomes misleading: on the reference test set a model that never predicts leakage would already be 69% accurate. Remedies act on the data, on the algorithm or on the decision threshold.

Resampling and SMOTE

Random undersampling discards majority samples, and random oversampling duplicates minority samples. The *Synthetic Minority Over-sampling Technique* (SMOTE) [20] instead creates new minority samples by interpolating between a minority sample $\mathbf{x}_i$ and one of its nearest minority neighbours $\mathbf{x}_{\text{nn}}$ (Equation (9)). *Edited Nearest Neighbours* (ENN) then removes samples whose neighbours mostly belong to the other class; the hybrid SMOTE+ENN was the best method in the study of Hakim et al. [11].

$$\mathbf{x}_{\text{new}} = \mathbf{x}_i + \delta\,(\mathbf{x}_{nn} - \mathbf{x}_i), \qquad \delta \sim U(0,1) \tag{9}$$

Two cautions are specific to this project. First, on binary and categorical attributes interpolation creates impossible values such as `Through_pin` = 0.37; variants such as SMOTE-NC and SMOTEN handle categorical features. Second, any resampling must be applied *only to the training data* inside each cross-validation fold; resampling before the split is synthesis leakage [3]. Resampling also cannot separate identical events with different labels, so it changes the operating point of the model rather than its ceiling.

Class weights and threshold moving

Cost-sensitive learning gives errors on the minority class a higher cost. In scikit-learn the "balanced" class weights follow Equation (10), where N is the number of samples, K the number of classes and N_k the number of samples of class k. *Threshold moving* keeps the model unchanged and lowers θ in Equation (1) so that more events are flagged; the threshold is chosen on validation data to balance recall against the number of alerts.

$$w_k = \frac{N}{K N_k} \tag{10}$$

Model Selection and Hyper-parameter Tuning

Hyper-parameters are chosen by comparing candidate settings, never on the test set. In *k-fold cross-validation* the training set is divided into k folds, each used once for validation while the others are used for training; the *stratified* variant keeps the class proportions in every fold. *Grid search* (GridSearchCV in scikit-learn [10]) tries every combination of predefined values, while *random search* samples combinations and often finds equally good settings faster. Three rules make the search valid for this project: the scoring function must reflect the goal (F1, recall or average precision of the leakage class, not accuracy); every data-dependent step must sit inside a `Pipeline` so that it is refitted on each training fold; and because the best cross-validation score is itself optimistically biased, the final model is evaluated once on an untouched test set.

Evaluation Metrics

Confusion matrix and basic metrics

Treating leakage as the positive class, a *true positive* (TP) is a leakage event correctly flagged, a *false positive* (FP) is a false alarm, a *false negative* (FN) is a missed leakage event and a *true negative* (TN) is a normal event correctly passed. Figure 8 redraws the

confusion matrix published in the reference article. *Accuracy* (Equation (11)) is the share of correct predictions; *precision* (Equation (12)) is the share of alerts that are real leakage, which measures wasted analyst time; *recall* (Equation (13)) is the share of leakage events detected; and the *F1-score* (Equation (14)) is their harmonic mean. *Specificity*, TN / (TN + FP), is the share of normal events correctly passed.

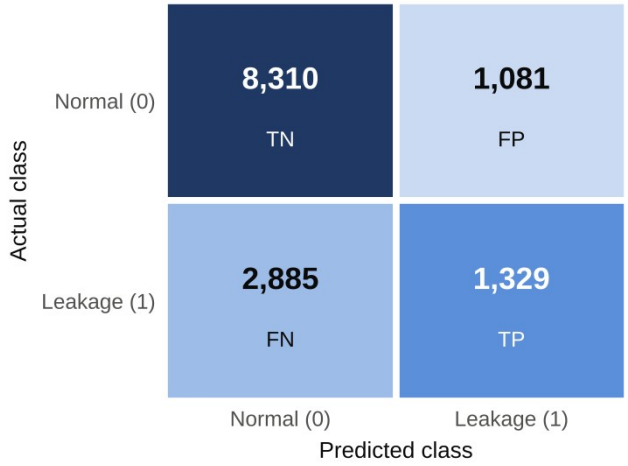


Figure 8: Confusion matrix of the optimised decision tree in the reference article

Redrawn from the values published in [9]. The 2,885 false negatives are leakage events that the model missed, more than twice the number it detected (1,329).

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \tag{11}$$

$$\text{Precision} = \frac{TP}{TP + FP} \tag{12}$$

$$\text{Recall} = \frac{TP}{TP + FN} \tag{13}$$

$$F_1 = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} \tag{14}$$

Metrics robust to imbalance

Balanced accuracy, the mean of recall and specificity, gives 0.5 to a majority-class predictor. The *Matthews correlation coefficient* (MCC) of Equation (15) uses all four cells; it ranges from −1 to +1, with 0 for random guessing, and is high only when the model does well on both classes. When a metric is computed per class, the *macro* average of Equation (16) weights every class equally, while the *weighted* average follows the class sizes n_k and is dominated by the majority class. Threshold-independent views complete the picture: the ROC curve plots recall against the false positive rate, while the precision-recall curve and its average precision are more informative on imbalanced data.

$$\text{MCC} = \frac{TP \cdot TN - FP \cdot FN}{\sqrt{(TP + FP)(TP + FN)(TN + FP)(TN + FN)}} \tag{15}$$

$$M_{\text{macro}} = \frac{1}{K}\sum_{k=1}^{K} M_k, \qquad M_{\text{weighted}} = \sum_{k=1}^{K} \frac{n_k}{n} M_k \tag{16}$$

Applying the metrics to the reference article

Table 12 applies these definitions to the confusion matrix of Figure 8. It explains why the same model can be described as having an F1-score of 0.68, 0.60 or 0.40 depending on the averaging, and shows that balanced accuracy and MCC give a far more sober picture than accuracy. These values are the baseline the project must improve on, always read against the performance ceiling discussed earlier.

Table 12: Metrics recomputed from the confusion matrix of the reference article

Metric	Formula	Value	Interpretation
Accuracy	Eq. (11)	0.708	Only 0.018 above the majority-class baseline (0.690)
Precision (leakage)	Eq. (12)	0.551	About 45% of alerts are false alarms
Recall (leakage)	Eq. (13)	0.315	About 68% of leakage events are missed
Specificity	TN / (TN + FP)	0.885	11.5% of normal events trigger an alert
F1 (leakage / normal)	Eq. (14)	0.401 / 0.807	Poor detection of the minority class
F1, macro / weighted	Eq. (16)	0.604 / 0.682	Weighted average hides the weak minority class
Balanced accuracy	(Recall + Specificity) / 2	0.600	Majority-class predictor would score 0.500
MCC	Eq. (15)	0.243	Weak positive correlation between predictions and labels

Computed from TN = 8,310, FP = 1,081, FN = 2,885 and TP = 1,329 published in [9].

Methodological Data Leakage in Machine Learning

Methodological leakage occurs when a model is trained or selected using information that would not be available in practice, so that the measured performance overestimates the real one [2], [3]. Kapoor and Narayanan found it in hundreds of published ML-based studies across many fields [2]. Apicella et al. [3] organise leakage by the pipeline stage at which it enters (Figure 9): *data-induced* leakage while the dataset is built (including oversampling before the split), *preprocessing-related* leakage when transformation parameters are estimated on data that include the test set, and *split-related* leakage when identical samples fall on both sides of the split or the split ignores the structure of the data.

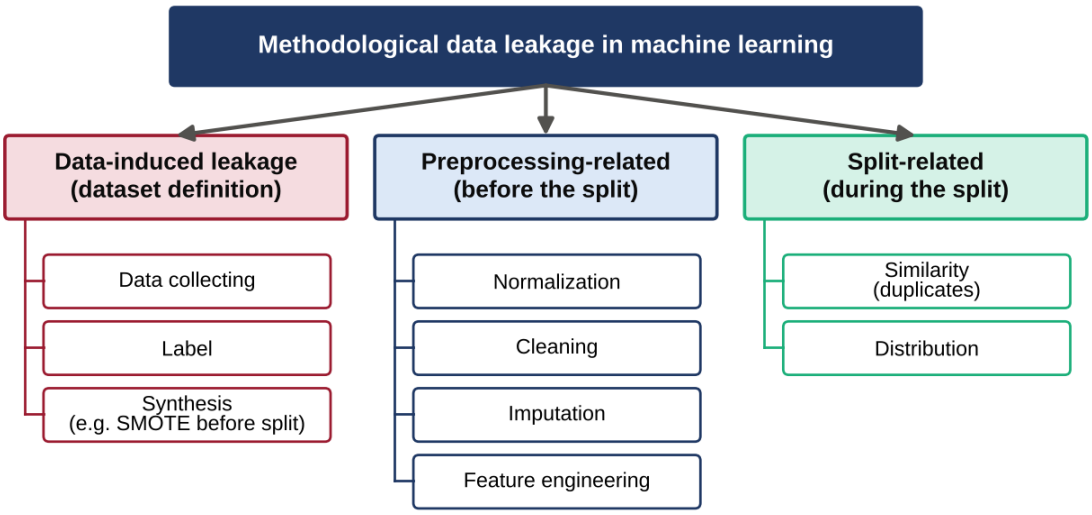


Figure 9: Taxonomy of methodological data leakage

Redrawn and simplified from the categorisation proposed by Apicella et al. [3].

Figure 10 contrasts a leaky ordering of steps with a safe one, and Table 13 lists how each type is handled in this project. The guiding rule is that the test set must play the role of future data the developer has never seen.

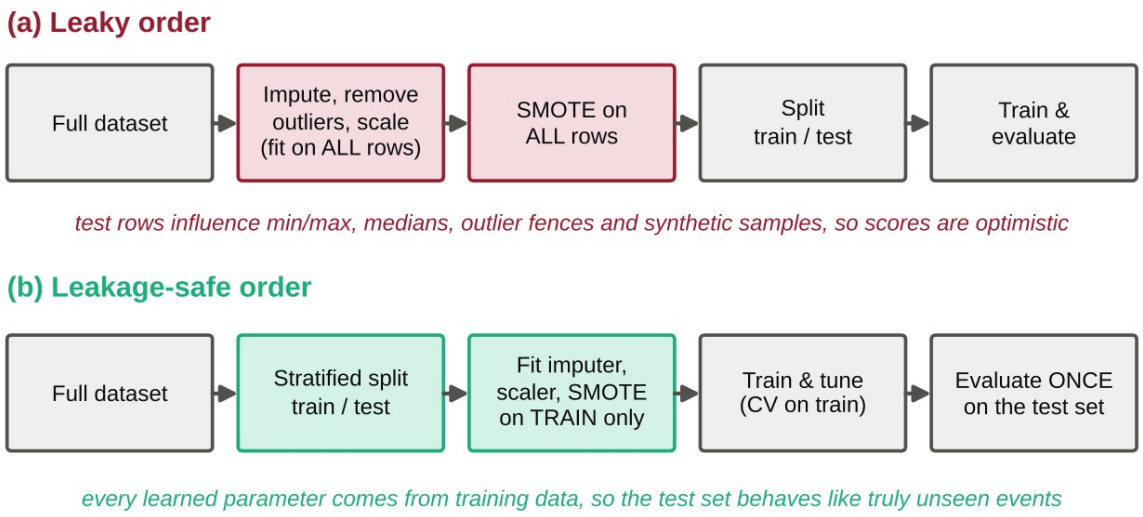


Figure 10: Leaky and leakage-safe orderings of pipeline steps

(a) Fitting preprocessing or SMOTE on all rows lets test information into training. (b) Splitting first and fitting on training data only keeps the test set independent.

Table 13: Leakage types and the safe practice adopted in this project

Leakage type	Risk in this project	Safe practice adopted
Synthesis	SMOTE or SMOTE+ENN applied before splitting	Resample inside each training fold through a pipeline
Normalisation / imputation	Min-max parameters or modes computed on all rows	Fit transformers inside the pipeline (or omit scaling for trees)
Cleaning	IQR fences computed on all rows; outliers removed from the test set	Compute fences on training data; never remove test rows
Feature engineering	Feature selection or target encoding on all rows	Perform selection and encoding inside cross-validation
Similarity	Identical feature vectors in training and test sets	Measure the overlap, report it and relate results to the ceiling
Model selection	Choosing the model or threshold on the test set	Choose on validation folds; use the test set once

Based on the safe practices proposed in [3] and adapted to the dataset and methods of the project.

From Model to Cloud Service

A detector is only useful if it can score real events as they happen. This section explains where such events come from in a cloud environment and how the project's prototype will connect a trained model to them.

Cloud-storage audit logs

Cloud providers record API activity in audit logs. In Amazon Web Services (AWS), each CloudTrail record contains, among other fields, the time of the request (`eventTime`), the service (`eventSource`), the action (`eventName`), the identity of the caller (`userIdentity`), the source IP address and the request parameters [21]. Object-level operations on Amazon S3, such as `GetObject`, `PutObject` and `DeleteObject`, are *data events* that are not logged by default and must be enabled explicitly [21]. When temporary credentials are used, the `userIdentity` element includes a `sessionContext` whose `mfaAuthenticated` attribute shows whether the user authenticated with an MFA device [21]. Other providers offer comparable audit logs. Table 14 shows how each attribute of the project dataset could be derived from such records.

Table 14: Possible mapping from cloud audit-log fields to the dataset attributes

Dataset attribute	Source in an AWS-style audit log	Remarks
`Authority`	Role or group of the principal in `userIdentity`, matched to the organisation's role list	Requires a lookup table from IAM roles to job roles
`Through_pwd`, `Through_MFA`	Sign-in method; `sessionContext.attributes.mfaAuthenticated`	MFA status is recorded directly
`Through_pin`	Sign-in details from the identity provider	No direct CloudTrail equivalent
`Operation`	`eventName`: `GetObject` (read), `PutObject` (write), `CopyObject` (move), `DeleteObject` (delete)	A move is usually a copy followed by a delete
`Data_Modification`	Write or delete events on an object	Derived from `eventName`
`Data_Sensitivity`, `Confidential_Data_Access`	Classification tags of the object named in `requestParameters`	Needs a data-classification step (DLP discovery)
`Confidential_File_Transfer`	Copy, download or share of a confidential object to an external destination	Needs the destination account or link type

The mapping is a design proposal for the prototype, not a feature of the Kaggle dataset, whose origin is not documented in the reference article.

Prototype architecture

Figure 11 shows the planned prototype. Audit events are converted into the nine attributes by a feature mapper and sent to a scoring API that wraps the trained pipeline and the chosen threshold. The API returns a risk score, a label and, for tree models, the rule that fired. Alerts are shown on a dashboard and can be forwarded to a SIEM, as Komka et al. did with Splunk [12]. Because no production cloud account is available, the prototype will replay the held-out test events as a stream in the same JSON structure, so that the whole path from event to alert can be demonstrated and its latency measured. Table 15 shows the planned request and response of the API.

Table 15: Planned request and response of the scoring API

Part	Field	Example	Meaning
Request	`event_id`	evt-000123	Identifier of the audit event
Request	`authority`	3	Role code of the user (Senior Manager)
Request	`through_pwd`, `through_pin`, `through_mfa`	1, 0, 0	Password only, no PIN, no MFA
Request	`operation`, `data_modification`	3, 0	Move without modification
Request	`confidential_data_access`, `confidential_file_transfer`	1, 1	Confidential object accessed and transferred
Request	`data_sensitivity`	3	High sensitivity
Response	`risk_score`	0.62	Estimated probability of leakage
Response	`label`	leakage	Decision at the chosen threshold
Response	`rule`	transfer = 1 AND access = 1 AND authority ≤ 3	Tree path that produced the decision

Field names follow the dataset attributes; the values are an invented example. The rule shown is the leakage path of the reference article's tree.

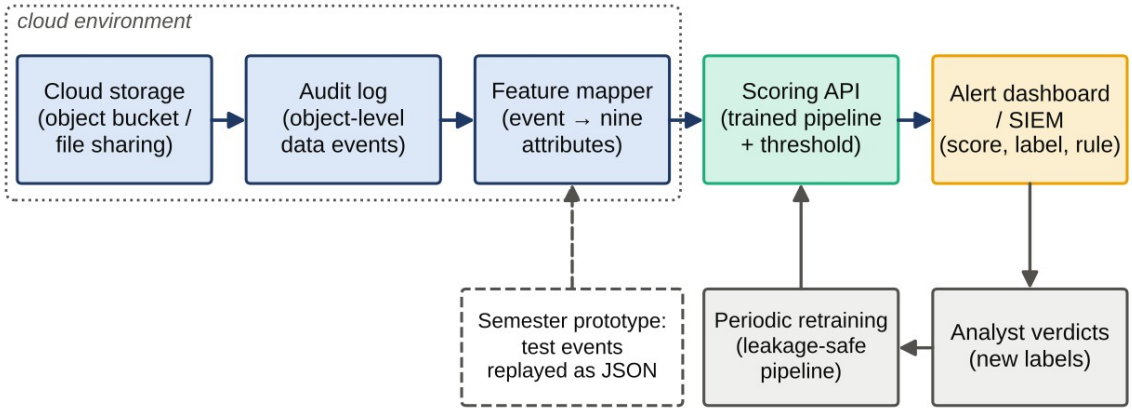


Figure 11: Planned prototype: from cloud-storage audit events to alerts

Blue blocks run in the cloud environment; the green block is the trained model exposed as an API; the yellow block is what analysts see. In the semester prototype, the test events are replayed through the same path.

Monitoring and retraining

Once deployed, a model degrades as user behaviour and attack techniques change, a phenomenon known as concept drift. Machine-learning systems also accumulate hidden maintenance costs in their data pipelines and configuration. The prototype will therefore log the distribution of scores and the daily alert rate, store analyst verdicts as new labels, and retrain the pipeline periodically with the same leakage-safe procedure.

Chapter Summary

This chapter introduced the project: its importance, the scientific problem, the reference article and the objectives that improve on it, three recent related studies, the practical applications and the main challenges. It then laid out the theory the project needs. On the security side, it explained why data leakage, and insider misuse in particular, is hard to stop with static rules, defined the threat model and its mapping to MITRE ATT&CK, and related DLP and access control to the dataset attributes. On the learning side, it showed that the coarse attributes impose a measurable performance ceiling, and covered decision trees, ensembles, imbalance, metrics and the safeguards against methodological leakage. Finally, it described how the detector can run as a service on cloud audit logs. The next chapter will implement this framework.

Chapter 2: Implementation Mechanism and Experimental Results

This chapter turns the framework of Chapter 1 into a working experiment. It first describes the implementation mechanism as a block diagram, two flowcharts and three algorithms in pseudo-code, then the experimental set-up, and finally the results, step by step, in the order in which they were obtained. The whole experiment is one Jupyter notebook, `leakage_detection.ipynb`, written in Python with scikit-learn [10], imbalanced-learn and XGBoost [19]. It runs from top to bottom in Google Colab or locally, and writes every table and figure of this chapter to its `results/` folder.

Implementation Mechanism

The mechanism is shown at three levels of detail that use the same stage names: the block diagram (Figure 12) shows the components, the flowcharts (Figure 13 and Figure 14) show the order of the decisions, and the pseudo-code (Algorithms 1 to 3) shows the exact procedure. Components that are planned but not yet implemented are drawn with dashed borders.

Notation

D is the labelled dataset of n = 49,500 events, $\mathbf{x}$ the eleven attributes of an event (the article's nine plus External Destination and the hour of the event) and y its label (1 = leakage). Acc^* is the performance ceiling of Equation (4) and c the share of events whose attribute combination appears with both labels. D_{tr} and D_{te} are the training and test subsets; $s(\mathbf{x})$ is the leakage score, θ the decision threshold and θ^* the threshold that is finally selected. r^* is the target recall for the leakage class (0.80), ε the tolerance within which the decision tree is preferred to a more complex model, and P^* the final trained pipeline.

Block diagram

Figure 12 groups the components into four stages. The first stage audits the dataset, computes the performance ceiling and splits the data before anything is fitted, so that 20% of the events are locked away as the test set. The second stage works on the training set only, always with the same stratified 5-fold cross-validation: the article's reference tree is rebuilt, External Destination and the hour are added, three class-weighted models are tuned, four imbalance strategies are compared, and the decision threshold θ^* is chosen from out-of-fold scores. The third stage uses the test set exactly once, compares the result with the majority baseline, the reference tree and the ceiling, and explains the selected model. The fourth stage, still planned, exports the model to the prototype service of Chapter 1, where audit events are mapped to the eleven attributes, scored and turned into alerts.

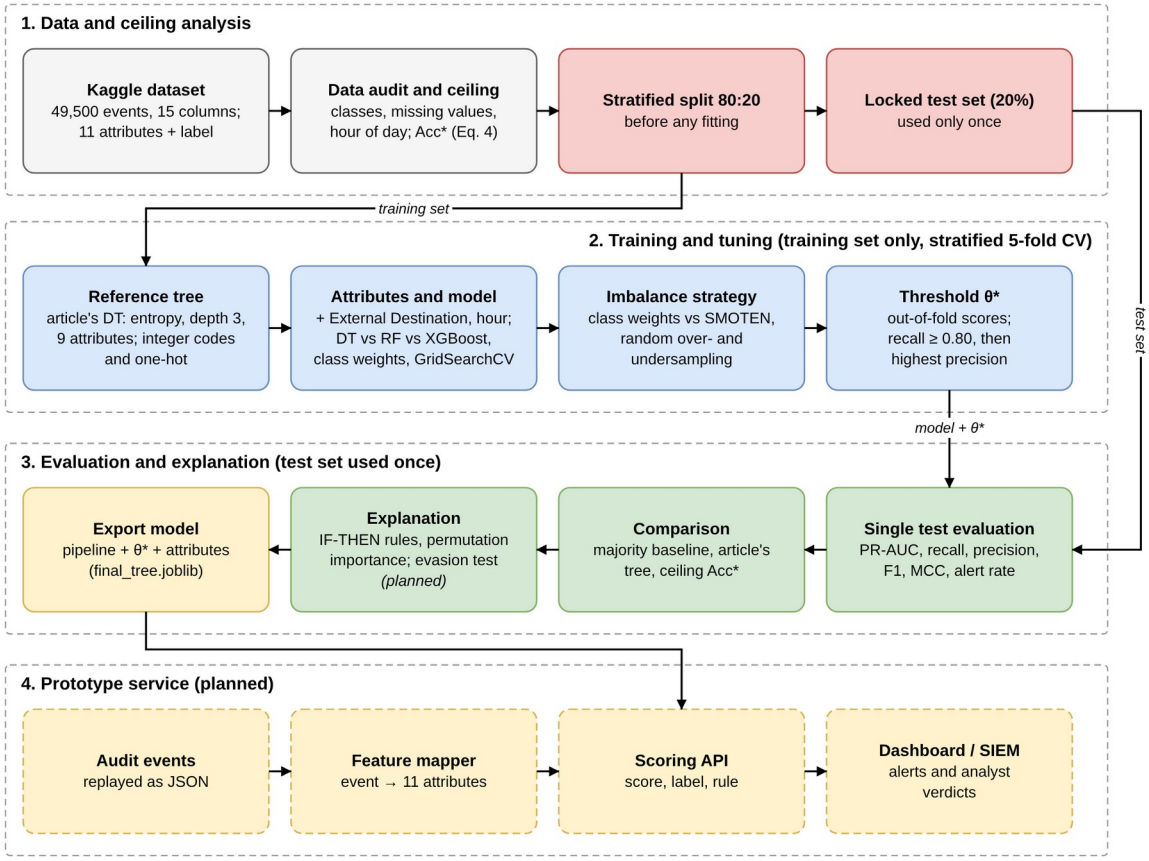


Figure 12: Block diagram of the implementation mechanism

Grey: data and ceiling analysis; red: split; blue: steps fitted on training data only; green: single test evaluation; yellow: model export and prototype service; dashed: planned, not yet implemented.

Flowcharts

The procedure has an offline part, in which the model is built and evaluated, and an online part, in which it scores events. Figure 13 shows the offline part. After the audit, the ceiling computation and the split, the article's tree (entropy, depth 3, nine attributes, one-hot encoding) is cross-validated as the reference. The decision tree is then tuned with class weights on the nine attributes, External Destination and the hour are added, and three class-weighted models (decision tree, random forest and XGBoost) are tuned with GridSearchCV on the eleven attributes. GridSearchCV refits the encoding and any other fitted preprocessing inside each training fold, so that neither validation nor test data influence them. The decision tree is kept when its PR-AUC is within ε of the best model. Four imbalance strategies are then compared on that tree, with resampling applied to the training folds only. The threshold is swept along the precision-recall curve of the out-of-fold scores: if some θ reaches the recall target r^*, the most precise such θ is kept; otherwise the threshold that maximises the leakage F1-score is used and the gap is reported. A table of operating points shows the analyst workload of other choices. Only then is the test set opened, once.

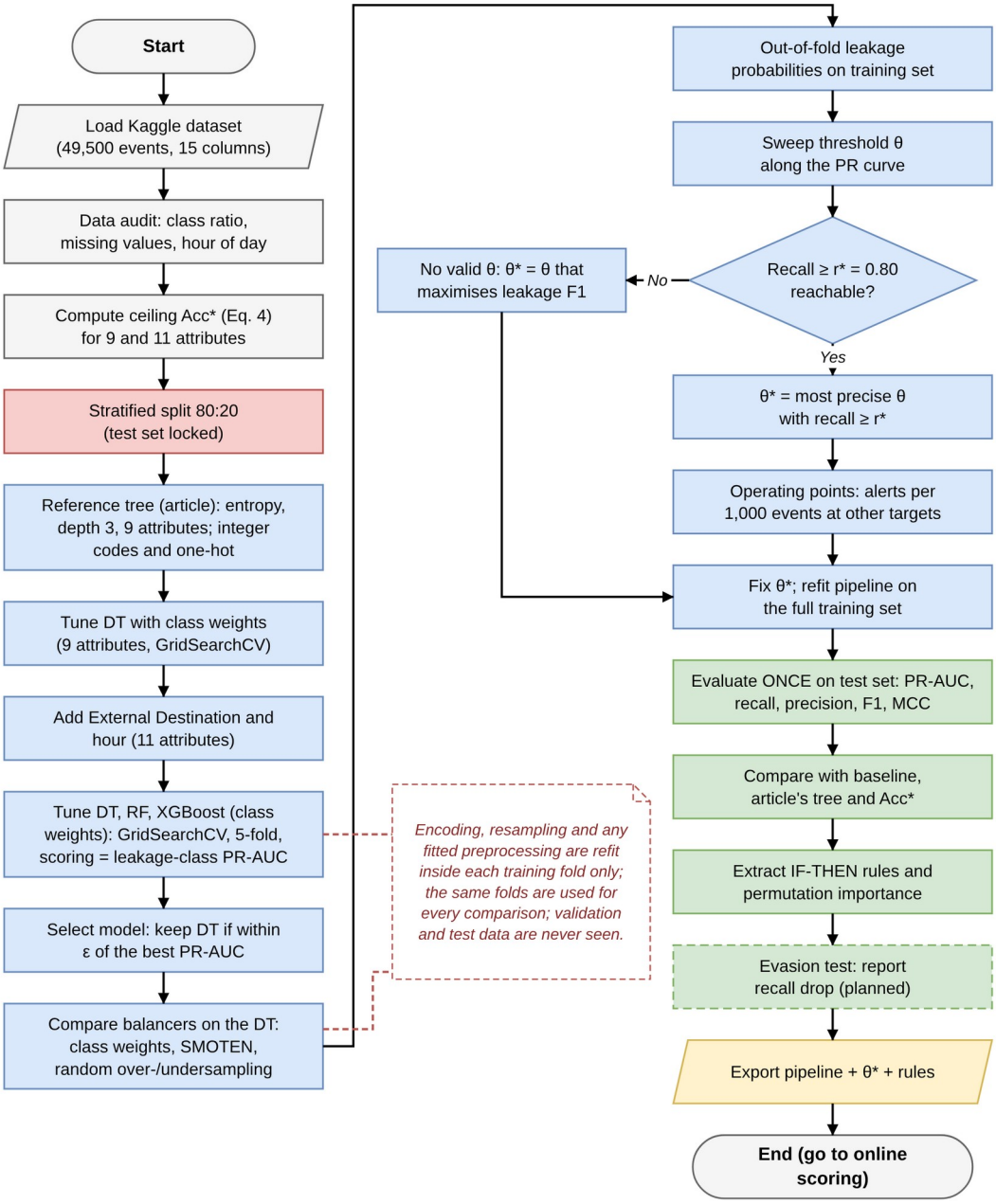


Figure 13: Flowchart of model training and evaluation (offline phase)

The left column prepares the data and runs the training-set experiments in the order of the notebook; the right column selects the threshold and then uses the test set once. Dashed: planned.

Figure 14 shows the online part, which belongs to the planned prototype. Each audit event is mapped to the eleven attributes, the hour being read from the event timestamp; events that cannot be mapped receive the provisional label *review* and are sent to an analyst instead of being scored. The risk score $s(\mathbf{x})$ is compared with θ^*: events above the threshold raise an alert with the rule that fired, and the analyst's verdict is stored as a new label. Score distributions, alert rates and latency are monitored, and retraining with the procedure of Figure 13 is triggered by drift or by a fixed period.

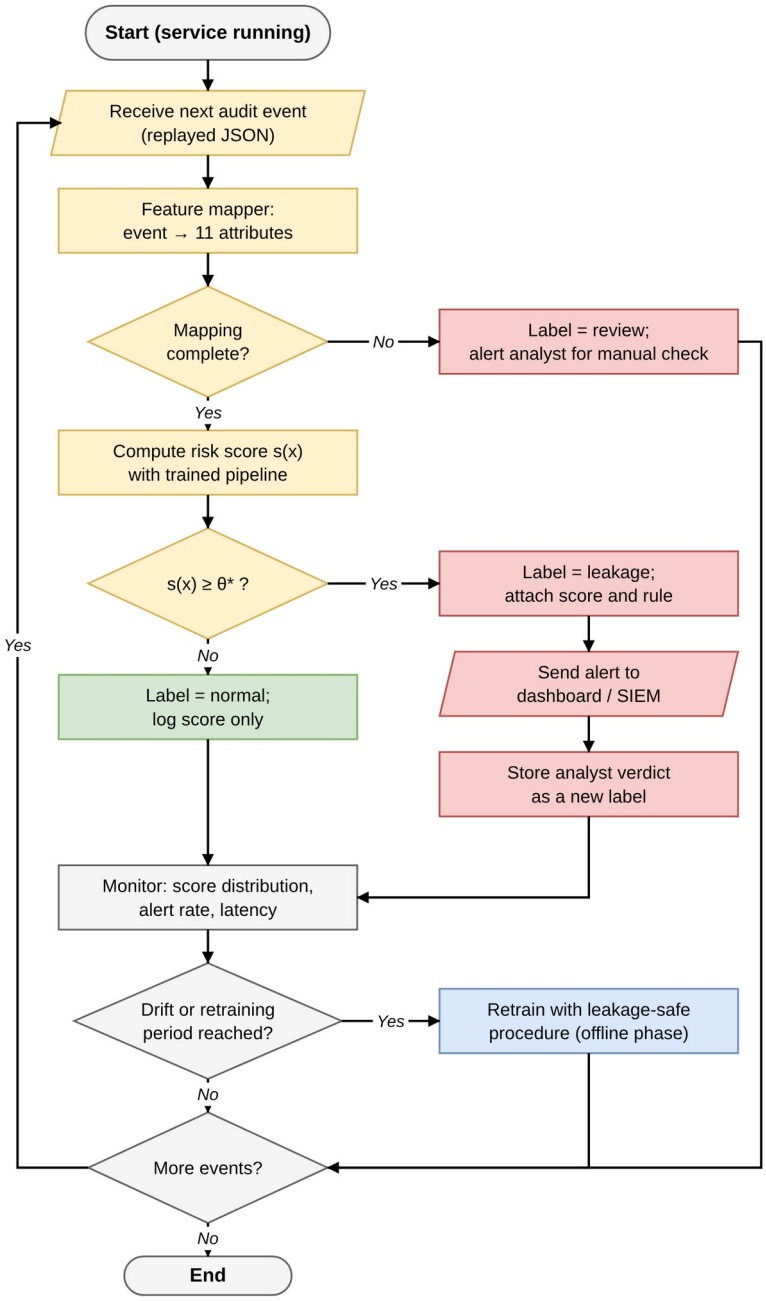


Figure 14: Flowchart of online scoring of audit events (planned prototype)

In the semester prototype, which is still under development, the audit events will be the held-out test events replayed as JSON.

Pseudo-code

Algorithm 1 computes the performance ceiling and the share c of events whose attribute combination carries both labels. Algorithm 2 is the leakage-safe training and evaluation procedure of Figure 13, in the order in which the notebook runs it; its comments mark the points at which methodological leakage is prevented. Algorithm 3 scores a single audit event as in Figure 14.

Algorithm 1: Performance ceiling (COMPUTE-CEILING)

```
Input : labelled dataset D = {(x_i, y_i)}, i = 1..n
Output: ceiling accuracy Acc*, share of conflicting events c,
        attribute combinations L ranked by leakage rate

 1: V ← set of distinct attribute combinations in D
 2: for each v in V do
 3:     n_v ← number of events with x = v
 4:     m_v ← number of those events with y = 1
 5: end for
 6: Acc* ← (1/n) · Σ_{v in V} max(m_v, n_v − m_v)            // Eq. (4)
 7: c ← (1/n) · Σ_{v in V : 0 < m_v < n_v} n_v               // same x, labels 0 and 1
 8: L ← V sorted by m_v / n_v in descending order
 9: return Acc*, c, L
```

Algorithm 2: Leakage-safe training and evaluation (TRAIN-AND-EVALUATE)

```
Input : dataset D; article attributes A9; A11 = A9 + {External Destination, hour};
        Models = {DT, RF, XGBoost}, each with class weights;
        Balancers = {class_weight, SMOTEN, random oversampling, random undersampling};
        hyper-parameter grids Grid[m]; recall target r* = 0.80; tolerance ε
Output: final pipeline P*, threshold θ*, rules R, test metrics

 1: (D_tr, D_te) ← StratifiedSplit(D, test_size = 0.2, seed = 42)  // D_te locked
 2: cv ← StratifiedKFold(5, shuffle, seed = 42)     // same folds for every comparison
 3: Ref ← DT(entropy, depth = 3) on A9, one-hot encoded             // article's tree
 4: CrossValidate(Ref, D_tr, cv)
 5: G9 ← GridSearchCV(Pipeline(Encoder, DT), Grid[DT] + class_weight, cv,
                      scoring = leakage-class PR-AUC);  G9.fit(D_tr[A9])
 6: for each A in {A9, A9 + External Destination, A9 + hour, A11} do
 7:     CrossValidate(Pipeline(Encoder, DT(G9.best_params)), D_tr[A], cv)
 8: end for
 9: for each m in Models do
10:     G[m] ← GridSearchCV(Pipeline(Encoder, m), Grid[m], cv, scoring = PR-AUC)
11:     G[m].fit(D_tr[A11])                         // encoder refit inside each fold
12: end for
13: if G[DT].score ≥ max over m of G[m].score − ε then m* ← DT else m* ← best m
14: for each b in Balancers do
15:     CrossValidate(Pipeline(Imputer, b, Encoder, m*), D_tr[A11], cv)
16: end for                                      // resampling on training folds only
17: P* ← G[m*].best_estimator with the best balancer           // class weights
18: s_oof ← CrossValPredictProba(P*, D_tr, cv)
19: Θ ← {θ : Recall(s_oof, θ) ≥ r*}
20: if Θ is not empty then
21:     θ* ← the θ in Θ with the highest Precision(s_oof, θ)
22: else
23:     θ* ← the θ that maximises leakage F1(s_oof, θ); report gap to r*
24: end if
25: Ops ← alerts per 1,000 events for recall targets 0.5, 0.6, ..., 0.9
26: P*.fit(D_tr)
27: ŷ ← [s(x) ≥ θ*] for every x in D_te     // first use of D_te; never used to fit
28: metrics ← PR-AUC, ROC-AUC, F1, Recall, Precision, Accuracy, MCC, alert rate
29: compare metrics with majority baseline, Ref and Acc*
30: R ← ExtractRules(P*);  I ← PermImportance(P*, D_te)
31: Save(P*, θ*, A11)                                  // models/final_tree.joblib
32: [planned] evasion test on modified leakage events; report recall drop
33: return P*, θ*, R, metrics
```

Algorithm 3: Online scoring of one audit event (SCORE-EVENT)

```
Input : audit event e (JSON), trained pipeline P*, threshold θ*
Output: response {risk_score, label, rule}; label is leakage, normal or review

 1: x ← MapFeatures(e)   // event → 11 attributes (hour from the timestamp)
 2: if x is incomplete then
 3:     alert analyst for manual check of e
 4:     return {risk_score = null, label = "review", rule = null}
 5: end if
 6: s ← P*.predict_proba(x)[leakage]
 7: if s ≥ θ* then       // Eq. (1)
 8:     label ← "leakage"; rule ← DecisionPath(P*, x)
 9:     send alert (s, label, rule) to dashboard / SIEM
10: else
11:     label ← "normal"; rule ← null
12: end if
13: log (e.id, s, label, latency) for monitoring
14: return {risk_score = s, label, rule}
```

Experimental Setup

The experiment uses the Kaggle file of the reference article [9]: 49,500 events and 15 columns. Its column names differ slightly from those in the article, and the role called *Expert Staff* in the article appears as *lead technician* in the file. Table 16 lists the eleven attributes used by the final model. The hour is derived from the `date` column, which comes in two text formats; only the hour is used, because the order of day and month in the second format is ambiguous while the hour is not.

Table 16: Attributes used in the experiments

#	Column in the Kaggle file	Name in the article	Values
1	`Authority`	Authority	staff, manager, senior manager, lead technician, intern
2–4	`Through_pwd`, `Through_pin`, `Through_MFA`	same	0 / 1
5	`Data Modification`	Data_Modification	0 / 1
6	`Confidential Data Access`	Confidential_Data_Access	0 / 1
7	`Confidential File Transfer`	Confidential_File_Transfer	0 / 1
8	`File Operation`	Operation	read, write, move, delete
9	`Data Sensitivity Level`	Data_Sensitivity	low, medium, high
10	`External Destination`	not used	internal, external
11	`hour` (from `date`)	not used	0 to 23
–	`Abnormality`	Label	0 = normal, 1 = leakage

Attributes 1 to 9 are those of the article. The columns id, user and pc are identifiers and are never used as attributes.

All random steps use the seed 42. The data are split 80:20 with stratification before anything is fitted, which gives 39,600 training events and 9,900 test events, 3,079 of them leakage. Every comparison uses the same stratified 5-fold cross-validation on the training

set, and models are selected by the PR-AUC of the leakage class, which, unlike accuracy, does not reward a model that ignores the minority class. Missing values are imputed with the most frequent value (median for the hour) and categories are one-hot encoded, both inside the pipeline so that they are refitted in every fold. The run reported here used Python 3.12, scikit-learn 1.8, imbalanced-learn 0.14 and XGBoost 3.4. Table 17 lists the steps of the notebook.

Table 17: Steps of the notebook and their outputs

Step	What it does	Saved output
1. Loading	Reads the CSV and derives the hour from both date formats	–
2. Data audit	Class ratio, missing values, leakage rate by hour	`leakage_by_hour.png`
3. Ceiling	Acc* and c for the article's attributes and with External Destination	`ceiling.csv`
4. Split and reference	Stratified 80:20 split; the article's tree cross-validated	–
5. Tuning	Grid search over the tree on the nine attributes, with and without class weights	–
6. Attributes	Adds External Destination and the hour	`attribute_experiments.csv`
7. Models and imbalance	Decision tree, random forest, XGBoost; four imbalance strategies	`model_comparison.csv`, `imbalance_strategies.csv`
8. Threshold	θ^* from out-of-fold scores; operating points	`operating_points.csv`
9. Final test	Single evaluation on the locked test set	`test_results.csv`, `test_evaluation.png`
10. Explanation	IF-THEN rules and permutation importance	`top_rules.csv`, `permutation_importance.csv`
11. Saving	Pipeline, threshold and run record	`final_tree.joblib`, `run_info.json`

Step 0 installs the packages in Colab and asks for the CSV file when it is missing. A full run takes about 15 to 20 minutes.

Experimental Results

Data audit

The file holds 34,105 normal events (68.9%) and 15,395 leakage events (31.1%). About 1% of the values are missing in every attribute, between 353 and 500 per column, and none in the label. A model that always predicts "normal" therefore already reaches an accuracy of 68.9%, only 1.9 points below the 70.84% reported in the article. Two columns that the article did not use carry a clear signal. Events sent to an external destination are leakage in 39.0% of cases, against 25.9% for internal ones. The hour matters even more: the leakage rate is 40.4% outside 09:00 to 17:59 and 15.8% inside it (Figure 15). Both values are known when the event happens and neither determines the label on its own, so they are genuine behavioural attributes and not target leakage.

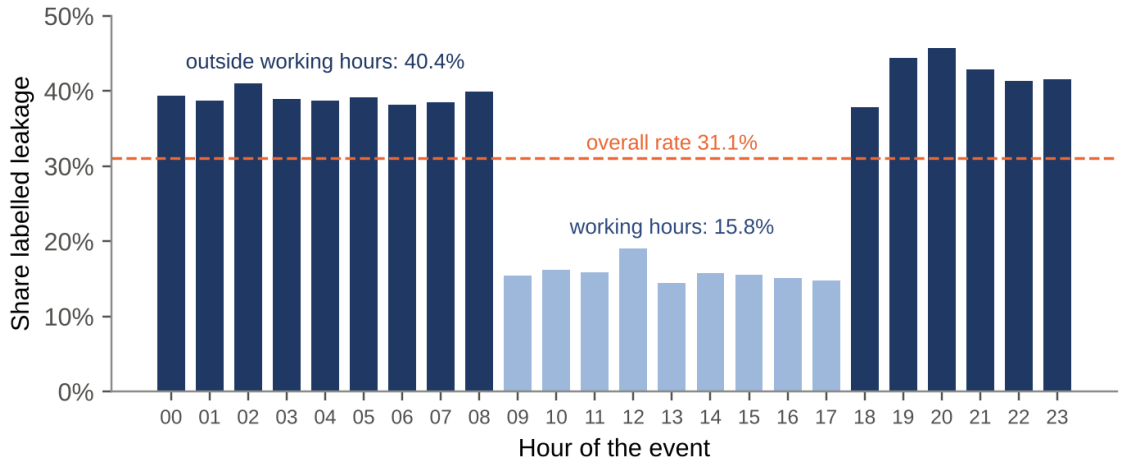


Figure 15: Leakage rate by hour of the event

Bars show the share of events labelled as leakage in each hour; the dashed line is the overall rate of 31.1%. Working hours (09:00 to 17:59) are shown in a lighter colour.

Performance ceiling

Algorithm 1 was applied to the full dataset, with a missing value counted as a category of its own (Table 18). With the article's nine attributes, 91% of the events share their exact attribute combination with events of the other label, and no model can exceed an accuracy of 74.3%. The whole room above the majority baseline is therefore 5.4 points, of which the article's 70.84% uses about a third. Adding External Destination raises the ceiling to 77.9%. The hour is not included in this calculation, because its 24 values split the data into combinations too small to be meaningful.

Table 18: Performance ceiling of the attribute sets

Attributes	Combinations	Majority accuracy	Ceiling Acc*	Conflicting share *c*
Nine attributes (article)	4,689	0.689	0.743	0.911
Nine + External Destination	6,737	0.689	0.779	0.850

The ceiling is optimistic, because a combination seen only once counts as correctly classified.

The reference tree under a leakage-safe evaluation

The article's tree was rebuilt with its published settings (entropy, maximum depth 3, the nine attributes and one-hot encoding, as stated in the article) and evaluated inside the leakage-safe pipeline. In cross-validation it reaches an accuracy of 0.691, barely above the majority baseline, and a leakage recall of 0.151, so it misses 85% of the leakage events (Table 19). It is also unstable across folds (F1 0.222 ± 0.083). Its PR-AUC of 0.443 is clearly above the leakage prevalence of 0.311: the tree ranks events reasonably well, but at the default threshold of 0.5 it almost never raises an alert.

Chapter 1 noted that the published tree splits on `Authority` ≤ 3.5, which is only possible when the roles are stored as integer codes. The same tree was therefore also rebuilt with integer codes. Read that way, it reaches an accuracy of 0.701 and a leakage recall of 0.298 on the test set, close to the 0.708 and 0.315 reported in the article. Both readings rank the events almost equally well (test PR-AUC 0.452 and 0.454). The one-hot reading, which

follows the text of the article, is used as the reference in the rest of the chapter, and the final comparison is also given against the integer-coded reading.

Tuning the tree on the nine attributes

A grid search over the criterion, the maximum depth, the minimum leaf size and the class weights selects an entropy tree of depth 10 with at least 20 events per leaf and balanced class weights, which make a leakage event count about 2.2 times as much as a normal one. On the same nine attributes, the leakage recall rises from 0.151 to 0.711 (Table 19); these are the early results of the project. Two further observations guided the next steps. First, PR-AUC is the same with and without class weights (0.4995 against 0.4994): weighting does not improve the ranking of events, it only moves the decision point, so the threshold is the main lever. Second, the eight best configurations lie within 0.002 PR-AUC of each other, so the exact settings matter little.

Table 19: Reference tree and tuned tree on the nine attributes (5-fold cross-validation)

Model	PR-AUC	F1	Recall	Precision	Accuracy
Reference tree (entropy, depth 3)	0.443	0.222	0.151	0.528	0.691
Tuned tree with class weights	0.500	0.552	0.711	0.451	0.641

Means over the five folds of the training set. Accuracy falls because the tuned tree raises more alerts.

Adding External Destination and the hour

With the tuned settings fixed, only the attribute set was changed (Table 20). Each new attribute alone adds about 0.05 PR-AUC, and together they add 0.122, from 0.500 to 0.622. At the default threshold the F1-score is the same with or without the hour (0.630 against 0.629); the gain from the hour lies in the ranking, which places the riskiest events first and matters when analysts can review only a limited number of alerts.

Table 20: Effect of the attribute set on the tuned tree (5-fold cross-validation)

Attributes	PR-AUC	F1	Recall	Precision	Accuracy
Nine attributes	0.500	0.552	0.711	0.451	0.641
+ External Destination	0.545	0.630	0.777	0.530	0.717
+ hour	0.558	0.582	0.710	0.493	0.683
+ External Destination + hour	0.622	0.629	0.731	0.553	0.732

Same tree, same folds; only the attributes change.

Model comparison

The decision tree was compared with a random forest [18] and with XGBoost [19], each tuned on the eleven attributes with class weights (Table 21). The ensembles gain only about 0.01 PR-AUC, against 0.122 from the attributes. XGBoost is the best model, 0.013 ahead of the tree, which is within the tolerance ε. The decision tree is therefore kept: its performance is almost the same, it produces readable IF-THEN rules, and it belongs to the model family of the article, so the comparison with the article remains fair.

Table 21: Comparison of the three models on the eleven attributes (5-fold cross-validation)

Model	Best hyper-parameters	PR-AUC	F1	Recall	Precision
Decision tree	entropy, depth 10, min. leaf 20	0.622	0.629	0.731	0.553
Random forest (200 trees)	depth 10, min. leaf 1	0.633	0.637	0.764	0.546
XGBoost (300 trees)	depth 6, learning rate 0.05	0.634	0.637	0.726	0.567

Each model was tuned by GridSearchCV with PR-AUC as the selection metric.

Imbalance handling

Four strategies were compared on the selected tree (Table 22). The resamplers run inside an imbalanced-learn pipeline after imputation and before encoding, so that they see complete original categories and only ever touch the training folds. Class weights are best or tied on every metric, need no synthetic data and are the fastest. SMOTEN [20] is the weakest: when most attribute combinations already carry both labels, synthetic leakage events add noise rather than information. The final model therefore keeps class weights and uses no resampling.

Table 22: Imbalance strategies on the selected tree (5-fold cross-validation)

Strategy	PR-AUC	F1	Recall	Precision
Class weights (balanced)	0.620	0.628	0.723	0.555
Random oversampling	0.612	0.625	0.705	0.562
Random undersampling	0.610	0.627	0.724	0.554
SMOTEN	0.601	0.609	0.678	0.553

The class-weight row differs from the previous table by 0.002 because the hour is imputed slightly differently in this pipeline.

Choosing the alert threshold

The threshold was chosen from out-of-fold scores on the training set, while the test set was still locked. The threshold with the highest F1-score is 0.535 (precision 0.577, recall 0.698). Requiring a recall of at least 0.80 and taking the most precise threshold that reaches it gives $\theta^* = 0.416$ (precision 0.508, recall 0.800), at a cost of only 0.011 in F1. Because a missed leak costs more than an extra alert, $\theta^* = 0.416$ was selected. Table 23 shows the full trade-off as it would appear to a security team: at a recall of 0.5 about two alerts in three are real leaks, at 0.9 fewer than one in two. If analyst capacity is the binding constraint, the rows for a recall of 0.6 or 0.7 are the better choice.

Table 23: Operating points of the final tree (out-of-fold scores, training set)

Recall target	Threshold	Precision	Alerts per 1,000 events	Real leaks per 1,000 alerts
0.5	0.739	0.681	229	681
0.6	0.639	0.640	292	640
0.7	0.529	0.575	379	575
0.8 (selected)	0.416	0.508	490	508
0.9	0.279	0.442	633	442

Each row is the most precise threshold that reaches the recall target.

Final evaluation on the test set

The test set of 9,900 events was opened once, after all choices had been fixed (Table 24, Figure 16). The test results confirm cross-validation (recall 0.827 against 0.800, F1 0.628 against 0.621), so the leakage-safe procedure gave an honest estimate. The final tree catches 2,547 of the 3,079 leakage events and misses 532. The reference tree catches 311, so the final tree detects about **eight times** as many leakage events; the MCC rises from 0.127 to 0.429 and the PR-AUC from 0.452 to 0.609. Against the integer-coded reading of the article's tree, which catches 918 leakage events, the factor is about 2.8, and against the recall of 0.315 reported in the article it is about 2.6.

Table 24: Results on the locked test set (9,900 events, 3,079 of them leakage)

Model	PR-AUC	F1	Recall	Precision	Accuracy	MCC	Alerts
Always "normal"	0.311	0.000	0.000	–	0.689	0.000	0%
Reference tree (article)	0.452	0.171	0.101	0.550	0.695	0.127	5.7%
Final tree (θ^* = 0.416)	**0.609**	**0.628**	**0.827**	0.507	0.696	**0.429**	50.8%

Precision is undefined for a model that never raises an alert. Alerts: share of test events flagged as leakage. ROC-AUC: 0.500, 0.688 and 0.809.

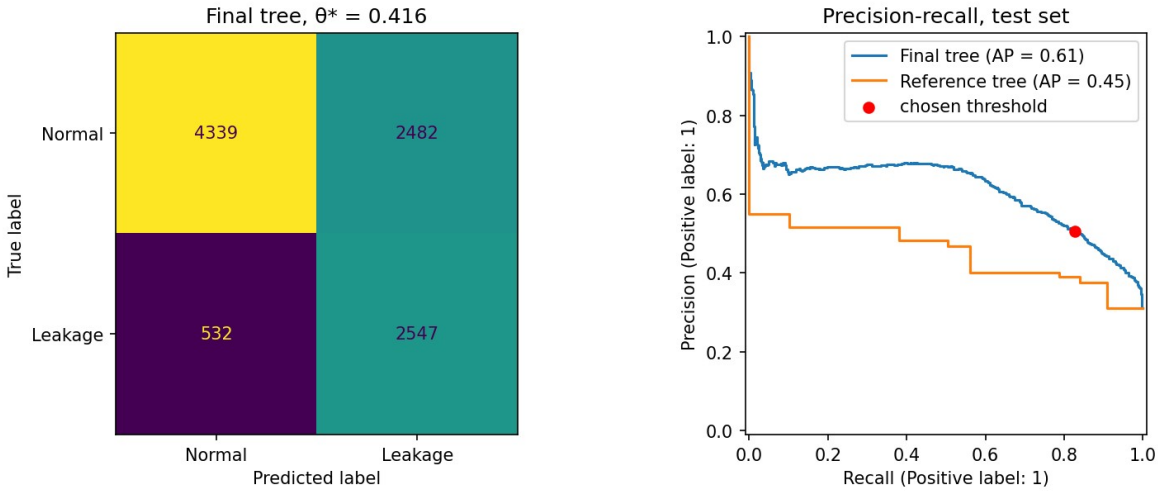


Figure 16: Confusion matrix of the final tree and precision-recall curves on the test set

Left: 2,547 leakage events are caught and 532 are missed, at the cost of 2,482 false alarms. Right: AP is the area under each curve (PR-AUC); the red point marks the selected threshold.

The two trees have almost the same accuracy, about 0.70, yet their recall differs by a factor of eight; accuracy alone hides the difference that matters in this task. The price of the higher recall is the alert volume: about half of all events raise an alert, and about one alert in two is a real leak (precision 0.507). This trade-off is deliberate, since a missed leak is more costly than a false alarm, and it is controlled by the threshold.

Interpretable rules and attribute importance

The final tree has 545 leaves, and 363 of them raise an alert at θ^*. Each alert can be explained by the path that produced it. Table 25 lists the five rules that catch the most leakage events in the training set. All of them begin with a confidential file transfer to an external destination, which agrees with the main rule of the article, and are then refined by

role, authentication and time of day: rules 1, 3 and 4 fire at night, with leakage rates between 0.61 and 0.72. With balanced class weights, a leaf raises an alert at θ^* when its training leakage rate is at least about 24%, below the base rate of 31%; this is why leaves with rates of 0.35 to 0.40, such as rules 2 and 5, still alert.

Table 25: The five rules that catch the most leakage events

#	Conditions after "confidential file transfer to an external destination"	Events	Leakage rate	Caught
1	role intern, senior manager or staff; no password; sensitivity high or medium; 00:00 to 02:59	369	0.61	225
2	role lead technician, manager, senior manager or staff; operation delete or read; no confidential data access; 09:00 to 14:59	601	0.35	209
3	role intern, manager, senior manager or staff; no password; sensitivity high or medium; operation delete, move or read; 03:00 to 04:59	249	0.72	180
4	role intern, manager, senior manager or staff; password but no PIN; no data modification; sensitivity high or low; operation move, read or write; 00:00 to 07:59	196	0.65	128
5	role lead technician, senior manager or staff; operation move; no confidential data access; 09:00 to 16:59	316	0.40	126

Counts refer to the training set. Together the five rules cover 7% of the training leakage events.

Permutation importance measures how much the test PR-AUC drops when one attribute is shuffled (Table 26). The two attributes that the article did not use, External Destination and the hour, rank second and fourth. Confidential data access and confidential file transfer remain essential, the three authentication flags have a moderate effect, and Data Modification contributes nothing.

Table 26: Permutation importance of the attributes on the test set

Attribute	Drop in PR-AUC	Standard deviation
Confidential Data Access	0.134	0.005
External Destination (new)	0.133	0.005
Confidential File Transfer	0.109	0.007
hour (new)	0.103	0.007
Through_pin	0.055	0.006
Through_MFA	0.053	0.004
Through_pwd	0.051	0.004
Authority	0.018	0.004
File Operation	0.017	0.004
Data Sensitivity Level	0.015	0.002
Data Modification	0.000	0.002

Ten repetitions per attribute. Importance shows a contribution to the model's performance, not a cause of leakage.

Discussion and Limitations

The experiment answers the questions raised by the reference article. Under a leakage-safe evaluation the article's tree is barely better than a model that never raises an alert, and it misses most leakage events. The largest improvement came from the information given to the model: two attributes added 0.122 PR-AUC, ten times more than replacing the tree with an ensemble, while class weights and the threshold turned that information into a high recall. The limits are equally clear:

- **Coarse attributes.** Even with External Destination, 85% of the events share their attribute combination with events of the other label, so precision stays around 0.5 at high recall.
- **Alert volume.** About half of all events raise an alert. A deployment would need fewer, richer alerts, for example from per-user behaviour history, which the dataset does not contain.
- **Length of the rules.** Each alert is explained by its path, but the rules have up to nine conditions; the global simplicity of the depth-3 reference tree was traded for detection performance.
- **Synthetic data.** The dataset is synthetic and limited to single events, so the results must be confirmed on real audit logs before any operational use.
- **Not yet tested against evasion.** The model relies strongly on the hour and the destination, which an informed insider could change. The evasion test and the prototype service are the next steps of the project.

Chapter Summary

This chapter described the implementation mechanism and the results of the project. A leakage-safe pipeline, in which the test set is locked before any fitting and every fitted step is refitted inside each fold, showed that the article's tree catches only 10% of the leakage events on the test set. A tuned, class-weighted decision tree with two additional attributes and a threshold chosen for a recall of at least 0.80 catches 83% of them, about eight times as many, with a PR-AUC of 0.609 and an MCC of 0.429, while every alert remains explainable by an IF-THEN rule. The remaining work is the evasion test and the prototype scoring service.

References

[1] T. M. Mitchell, *Machine Learning*. New York, NY, USA: McGraw-Hill, 1997.

[2] S. Kapoor and A. Narayanan, "Leakage and the reproducibility crisis in machine-learning-based science," *Patterns*, vol. 4, no. 9, art. no. 100804, 2023.

[3] A. Apicella, F. Isgrò, and R. Prevete, "Don't push the button! Exploring data leakage risks in machine learning and transfer learning," *Artificial Intelligence Review*, vol. 58, art. no. 339, 2025, doi: 10.1007/s10462-025-11326-3.

[4] Gartner, "Gartner forecasts worldwide public cloud end-user spending to total $723 billion in 2025," press release, Nov. 19, 2024. [Online]. Available: https://www.gartner.com/en/newsroom

[5] Cloud Security Alliance, "Top Threats to Cloud Computing 2024," Aug. 2024. [Online]. Available: https://cloudsecurityalliance.org

[6] IBM, "Cost of a Data Breach Report 2026," IBM Corporation, 2026. [Online]. Available: https://www.ibm.com/reports/data-breach

[7] TechRepublic, "New Verizon report reveals the security gap attackers are exploiting most" (summary of the Verizon 2026 Data Breach Investigations Report), 2026. [Online]. Available: https://www.techrepublic.com/article/news-verizon-dbir-vulnerability-exploitation-2026/

[8] S. Alneyadi, E. Sithirasenan, and V. Muthukkumarasamy, "A survey on data leakage prevention systems," *Journal of Network and Computer Applications*, vol. 62, pp. 137–152, 2016.

[9] P. Harahap and M. S. Hasibuan, "Detecting data leakage in cloud storage using decision tree classification," *Journal of Information Systems and Informatics*, vol. 7, no. 3, pp. 2516–2534, Sep. 2025, doi: 10.51519/journalisi.v7i3.1215.

[10] F. Pedregosa *et al.*, "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[11] A. R. Hakim, K. Ramli, M. Salman, B. Pranggono, and E. R. Agustina, "ARKAIV: Predicting data exfiltration using supervised machine learning based on tactics mapping from threat reports and event logs," *IEEE Access*, vol. 13, pp. 28381–28397, 2025, doi: 10.1109/ACCESS.2024.3524502.

[12] M. C. Komka, A. K. Ademuwagun, S. M. Abdullahi, T. M. Mashkur, and F. D. Ifeanyi, "Proactive insider threat detection in cloud environments using behavior-aware machine learning models," *Nature Journal of Emerging Sciences, Technologies, & Innovations*, vol. 8, no. 4, pp. 397–410, May 2026, doi: 10.65752/jr025522.

[13] P. Mell and T. Grance, "The NIST definition of cloud computing," National Institute of Standards and Technology, Gaithersburg, MD, USA, NIST SP 800-145, 2011.

[14] D. M. Cappelli, A. P. Moore, and R. F. Trzeciak, *The CERT Guide to Insider Threats*. Upper Saddle River, NJ, USA: Addison-Wesley, 2012.

[15] The MITRE Corporation, "MITRE ATT&CK: Enterprise matrix," ATT&CK v19, 2026. [Online]. Available: https://attack.mitre.org

[16] T. Hastie, R. Tibshirani, and J. Friedman, *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*, 2nd ed. New York, NY, USA: Springer, 2009.

[17] L. Breiman, J. H. Friedman, R. A. Olshen, and C. J. Stone, *Classification and Regression Trees*. Belmont, CA, USA: Wadsworth, 1984.

[18] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.

[19] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, 2016, pp. 785–794.

[20] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "SMOTE: Synthetic minority over-sampling technique," *Journal of Artificial Intelligence Research*, vol. 16, pp. 321–357, 2002.

[21] Amazon Web Services, *AWS CloudTrail User Guide* (sections "CloudTrail record contents", "Logging data events" and "CloudTrail userIdentity element"). [Online]. Available: https://docs.aws.amazon.com/awscloudtrail/latest/userguide/